

WEAVER: Foundation Model Training over AI-RAN Compute Infrastructure

Abstract

The emergence of AI-RAN architectures for future 6G networks envisions deep integration between radio access network (RAN) infrastructure and AI workloads. While prior work primarily focuses on AI-for-RAN and AI-on-RAN, the potential of AI-and-RAN – sharing RAN compute infrastructure with non-RAN (AI) workloads – remains largely unexplored. In this paper, we investigate the feasibility of leveraging GPU-accelerated RAN infrastructure to support opportunistic training of foundation models (FMs), one of the most compute-intensive AI workloads. We conduct the first comprehensive characterization of spare GPU capacity in GPU-accelerated RAN systems at both micro-scale (per-slot within a cell site) and macro-scale (across sites). Our analysis reveals substantial underutilized compute resources—approximately 40–85%—that are temporally bursty within sites yet spatially complementary across sites. To efficiently exploit this opportunity in a RAN-safe manner, we present WEAVER, a system that enables FM training to opportunistically share GPU resources with latency-critical RAN workloads while preserving RAN performance. WEAVER adopts a RAN-first design with coordinated elasticity across system layers. It introduces a RAN-centric spare-compute controller embedded in the MAC scheduler to stabilize spare GPU capacity, alongside a two-level elastic FM training framework that adapts to dynamic resources across and within sites. Evaluations on an O-RAN-aligned 5G NR prototype and multi-site testbed show that WEAVER reduces spare-compute fluctuations by up to 4.9× and achieves 2.1–4.2× higher training throughput than baseline approaches while harvesting up to 83% of available spare compute.

1 Introduction

As we head towards 6G, there is wide recognition that AI for radio access network (RAN) — AI-for-RAN — can unlock significant spectral, energy, and operational efficiency gains [52]. Building on this, there is growing momentum toward a more holistic integration of AI with the RAN to realize AI-RAN [30], which additionally aims to maximize RAN asset utilization and generate new revenue streams, as envisioned by the AI-RAN Alliance [2]. Such full integration involves transitioning to accelerated RAN compute hardware that can natively support AI workloads. Recent developments in this direction include the emergence of NVIDIA AI Aerial platforms [34] and the recently announced Nokia partnership with NVIDIA [33]. This would enable the full scope of AI-RAN, going beyond AI-for-RAN to also cover

“AI-on-RAN” to enable novel edge (AI) services, as well as “AI-and-RAN” to share RAN compute infrastructure between RAN and non-RAN (AI) workloads.

In this paper, we focus on the AI-and-RAN aspect of AI-RAN, that is, sharing RAN infrastructure between RAN and other workloads. This area has received relatively less attention to date than AI-for-RAN [15, 19, 49, 58] and AI-on-RAN [27, 63]. We use foundation model (FM) training as the non-RAN AI workload because it is arguably the most demanding in terms of compute requirements. FMs [6] are models trained at scale on broad data that can then be adapted to a wide range of downstream tasks (e.g., GPT-4 [41], Stable Diffusion [47], AlphaFold [28]). Large language models (LLMs) such as GPT-4 are a prominent subclass of foundation models trained on language data. FM training requires enormous compute. For example, for LLMs, exponential increases in model size, data size, and epochs lead to linear accuracy gains, which drives exponential growth in compute demand [29]. There is growing interest in developing domain-specific FMs, including for mobile networks [68], which further amplifies the compute resources needed to train them. This has contributed to a highly *centralized*, cloud-centric AI ecosystem.

We believe emerging AI-RAN infrastructure presents a compelling and timely opportunity for FM training for several reasons. First, the geographically distributed set of sites that make up the RAN infrastructure can serve as an alternative *decentralized* training platform. Second, the shift toward GPU-accelerated RAN infrastructure for AI-native 6G networks makes it well suited to AI workloads, including training. Third, the aggregate GPU compute capacity across RAN sites could potentially rival the currently deployed global GPU compute capacity¹. Fourth, because RAN compute usage depends on the traffic it carries, the inherent spatiotemporal dynamics of RAN traffic [24, 64, 65] create opportunities for spare compute, as RAN compute infrastructure is typically provisioned for peak usage (worst case).

Capitalizing on this opportunity, however, requires addressing **three main challenges**:

C0: First, we need to clearly understand the scale and nature of the spare compute likely to be available on an AI-RAN compute infrastructure (GPU-accelerated RAN). As discussed in

¹There are currently around 20 million 4G/5G RAN cells globally [43]. Assuming the number of cells remains at least at that level going forward to 6G, deploying a GH200 server (each with about 1 PFLOP capacity) for every 20 cells, as per the benchmarking in [39], yields an estimated total of 1 ZFLOP across all RAN sites. Current deployed GPU capacity globally is around 4 ZFLOPS [16].

§3.2 and §4, prior work provides limited insight, both in the scope of metrics and in the spatiotemporal scales considered.

C1: Second, even when spare compute exists, making it usable for non-RAN workloads such as FM training is challenging. This is because RAN traffic is inherently bursty at microscopic scales [18], which in turn manifests as burstiness in RAN related GPU compute usage. Smoothing that burstiness to make the spare GPU compute usable must be done carefully to ensure RAN performance is unaffected.

C2: Third, using the available spare GPU compute resource for FM training within and across cell sites over a distributed AI-RAN compute infrastructure requires a tailored training strategy that is robust and efficient in the face of resource dynamism and heterogeneity across sites and timescales.

In this paper, we start by tackling challenge **C0** by conducting a comprehensive characterization study on spare GPU compute capacity in GPU-accelerated RAN infrastructure for the first time, both at the micro-scale RAN slot level within a cell site and at the macro-scale across cell sites (§4). We find that substantial unused GPU resources (overall, about 40-85%) exist that could be harnessed by non-RAN workloads. This is especially the case for compute-intensive workloads like FM training that complement memory bandwidth-limited RAN processing. However, spare compute is temporally bursty at individual cell sites, while being spatially complementary across different sites.

Building on the above spare-compute characterization, we next address challenges **C1** and **C2**. To that end, we propose a system design termed **WEAVER** that enables FM training to opportunistically share GPU resources with RAN workloads on GPU-accelerated RAN infrastructure while guaranteeing that RAN performance remains unaffected. Its novelty lies in a RAN-first architecture with coordinated elasticity across system layers. **WEAVER** comprises two key components. First, **WEAVER** introduces a RAN-centric spare compute controller embedded inside the RAN stack, in contrast to relying on externally predicting RAN-related compute demand as done by prior works [50]. By integrating with the MAC scheduler, it performs compute-aware scheduling that smooths traffic spikes, enforces per-slot deadlines to guarantee RAN performance, and produces stable spare GPU compute that can be efficiently exploited by non-RAN workloads.

Second, **WEAVER** introduces a tailored approach for robust and efficient FM training over GPU-accelerated RAN infrastructure. Specifically, we propose a two-level elastic FM training strategy that adapts to fluctuating spare compute: at the inter-site level, a centralized coordinator dynamically redistributes training and selectively reconfigures model layouts across cell sites only when long-term compute shifts occur; at the intra-site level, training kernels are fine-grained and elastic, sharding general matrix multiplication (GEMM) operations (that dominate FM training) in real time

to match instantaneous spare GPU resource. A lightweight publish-subscribe interface couples these two components, allowing FM training jobs to continuously adapt to the RAN-provided spare GPU compute resources. Together, these design choices enable safe, efficient co-location of latency-critical RAN processing and opportunistic FM training.

We conduct a comprehensive evaluation of **WEAVER** that covers its two main components. With **WEAVER**'s RAN-centric spare-compute control deployed on an O-RAN-aligned 5G NR stack on DGX Spark, with GPU-accelerated LDPC decoding, the compute-aware MAC scheduler produces a stable spare-GPU compute resource, maximizing its utility for non-RAN workloads. Using production traffic traces and over-the-air (OTA) validation, we show that **WEAVER** reduces spare-compute fluctuations by up to 4.9 $\times$, while ensuring both RAN slot-deadline compliance and end-to-end UE performance. Using a prototype implementation of **WEAVER**'s tailored FM training system alongside its RAN-centric control over an 8-site cellular RAN testbed with an L40S GPU at each cell site, **WEAVER** achieves 2.1–4.2 $\times$ higher training throughput than baseline training systems, while harvesting 83% of available spare compute. We complement this testbed-based evaluation with a simulation study that trains larger models over larger RAN scenarios. Simulation results show that **WEAVER** achieves 2–7 $\times$ higher throughput and 3.4 $\times$ scaling efficiency in both number of sites/GPUs and model size, compared to the best alternative from the literature.

In summary, we make the following key contributions:

- **Spare compute characterization in AI-RAN (§4).** We present the first multi-scale characterization of spare GPU capacity in GPU-accelerated RAN infrastructure, showing 40–85% unused compute with temporally bursty but spatially complementary availability across cell sites.
- **WEAVER: RAN-first GPU compute sharing (§5 & §6).** We design **WEAVER**, a system that safely co-locates foundation model training with RAN workloads by introducing a RAN-centric spare compute controller integrated with the MAC scheduler to smooth GPU demand and expose stable spare compute without impacting RAN performance.
- **Elastic FM training for dynamic RAN resources (§7).** We develop a two-level elastic training strategy that adapts to fluctuating resources via inter-site workload redistribution and fine-grained intra-site sharding of training computations to match with spare GPU compute resource availability.
- **Comprehensive evaluation (§8).** Implemented on an O-RAN-aligned GPU-accelerated 5G stack, **WEAVER** reduces spare-compute variability by 4.9 $\times$, harvests 83% of spare GPU capacity, and improves training throughput by 2.1–4.2 $\times$, all while preserving RAN performance.

2 Background

RAN compute requirements. Virtualized RANs (vRANs) move baseband processing from proprietary hardware to general-purpose platforms [7, 18]. The Distributed Unit (DU) handles the most computationally demanding MAC and high PHY functions, with Forward Error Correction (FEC) via LDPC decoding as the dominant bottleneck [10, 51]. LDPC decoding must complete within 1–3 ms end-to-end to prevent synchronization loss and call drops, while operators simultaneously target carrier-grade service availability (often five nines, 99.999%). Meeting these latency and availability requirements purely on commodity CPUs is fundamentally difficult, motivating the use of dedicated hardware accelerators (HAs) in production vRAN deployments [10].

GPU-based acceleration of vRAN processing. Historically, ASICs and FPGAs have served as HAs for LDPC and other hot-path PHY tasks, but GPUs are increasingly attractive due to their massive parallelism and software programmability [50]. NVIDIA’s SionnaRK [10] targets GPU-accelerated LDPC decoding, while their Aerial platform [36, 37] offloads the entire 5G Layer-1 (PHY) pipeline processing to GPUs. These systems demonstrate that GPUs are an effective platform to host the compute-intensive PHY operations.

GPU architecture. A modern NVIDIA-style GPU consists of tens to hundreds of *Streaming Multiprocessors (SMs)*, each hosting hardware threads and execution pipelines, while all SMs share a high-bandwidth global memory [35]. The GPU programming model exposes these resources via *kernels*, which are functions launched by the host CPU, each with a specified SM demand. Matrix-heavy kernels (e.g., general matrix multiplication (GEMM) operations) tend to be *compute bound*, i.e., they saturate the arithmetic pipelines, whereas vector-heavy kernels (e.g., LDPC) are typically *memory-bandwidth bound*, i.e., they saturate memory bandwidth.

GPU utilization metrics. GPU load can be characterized with three metrics: (i) **SM Utilization (SMU)**: the fraction of SMs assigned to a kernel at a given time; (ii) **Arithmetic Compute Utilization (ACU)**: the fraction of time the SMs’ arithmetic pipelines actively issue instructions, normalized by the peak sustained issue rate; (iii) **Global Bandwidth Utilization (GBU)**: the ratio of achieved memory throughput to peak global memory bandwidth.

Foundation model training. FM training involves distributed computation over a directed acyclic graph (DAG) of dense matrix multiplications (GEMMs) [12, 55]. Each layer in the model produces a sequence of GEMM operations—arising from attention projections, feed-forward transforms, and embedding lookups—whose execution order and synchronization points are determined by the training algorithm (e.g., data parallelism (DP), pipeline parallelism (PP),

Table 1: Types of GPU sharing in current systems.

Paradigm	Mechanism	Isolation	Reconfiguration
Temporal	Context switch [17]	Device exclusive	Restore ~100 ms.
	Preemption [23, 54]	Instruction-level	Application rebuild
Spatial	MIG [71]	SM & memory	GPU reset ~7 s.
	Green Contexts [35]	SM	CUDA Stream ~10 μ s.
Mixed	MPS [51]	Best-effort SM (no mem. isol.)	GPU reset ~300 ms.
	CUDA streams [35, 57]	Best effort	Static ~1 μ s

or their combination). Over 99 % of training FLOPs for FMs are GEMMs [12, 55], which are compute-bound and saturate the GPU’s arithmetic pipelines rather than its memory bus.

3 Related Work

3.1 RAN Infrastructure Sharing

Here, we focus on the sharing of vRAN compute resources with third-party workloads. Inter-DU resource sharing (e.g., CloudRIC [51]) within the same server is an orthogonal problem and thus considered out of scope.

Sharing on CPU platforms. Existing works use core pinning and reallocation [18], cgroup-based CPU quota adjustment [40], and dynamic library redirection [25]. These mechanisms rely on fine-grained thread preemption and migration that GPUs lack, so they do not transfer to the GPU domain.

Sharing on GPU platforms. YinYangRAN [50] uses Multi-Process Service (MPS) to partition GPU SMs between 5G DU PHY processing and co-located ML inference, but MPS reconfiguration incurs non-negligible overhead (~0.3 s) during which the GPU is not usable and all workloads must fall back on CPUs. CAORA [53] instead partitions a single GPU into fully isolated Multi-Instance GPU (MIG) sub-instances with even larger reconfiguration overhead (~7 s) and coarse partition granularity. Both approaches rely on RAN workload prediction, which can be problematic, as overestimation starves the co-located workloads, while underestimation risks violating the PHY processing deadlines.

General GPU sharing mechanisms. Beyond MPS and MIG, modern GPUs expose other sharing mechanisms [35] that fall into three types (see Table 1); prior GPU systems mainly employ the temporal and mixed type mechanisms. However, they cannot be directly applied to RAN–AI coexistence for three reasons: (i) *fairness criterion differs*: these systems target fair sharing among tenants all deemed equal [13, 17, 71], whereas RAN must be unconditionally prioritized; (ii) *time granularity differs*: they operate at millisecond-to-second scheduling intervals [22], whereas RAN requires per-slot (≤ 1 ms) deadline guarantees; and (iii) *the isolation guarantees differ*: some systems distinguish a primary or latency-sensitive tenant from best-effort consumers [23, 54, 57] but still lack hard isolation and deadline guarantees for the primary workload, which are essential in the RAN context.

3.2 Characterizing RAN GPU Utilization

Opportunistic coexistence of non-RAN workloads (e.g., AI training) with latency-critical vRAN processing on shared GPUs requires accurate visibility into how RAN workloads consume GPU resources across GPU subsystems, time, and cell sites. Existing studies like CloudRIC [51], ETHOS [62], and DecodeX [46] profile the GPU utilization as a single scalar value, which does not accurately capture the GPU’s resource utilization in a fine-grained manner, e.g., SMU, GBU, and ACU, which are crucial for reliable workload sharing, as we show in WEAVING. Additionally, while CloudRIC studies the GPU’s utilization under different loads, it omits multi-DU setups as well as the macro-scale aspects, i.e., temporal and spatial cross-site utilization patterns, even though these macro-scale aspects are critical for understanding GPU utilization in real large-scale multi-site deployments.

3.3 ML Training over Dynamic Resources

Resource harvesting. While broadly relevant, volunteer computing [3, 4] and harvested cloud VMs [72] aggregate opportunistic idle cycles for embarrassingly parallel workloads but treat spare capacity as a binary idle-or-busy signal and scavenge slack at coarse (hourly) timescales. Crucially, they ignore the fine-grained structural, temporal, and spatial dimensions that govern GPU-accelerated RAN deployments. Also, they do not provide performance guarantees or cross-site coordination for tightly-coupled FM training.

Dynamic ML training systems. There exist training systems [11, 14, 67, 69] that adapt to resource dynamics but are not designed for the RAN setting and assume dedicated GPU access at each device, which contradicts sub-millisecond resource sharing with a high-priority, latency-critical co-tenant. Although they handle cross-site heterogeneity and churn, they do so relying on heavyweight mechanisms – coarse-grained checkpoint-and-restart (e.g., Bamboo [59], Mario [31]) or recomputation (e.g., SWARM [48], Asteroid [66]) – that slow down training when used in a highly dynamic RAN environment. Moreover, per-device AllReduce in DTFM [67] and communication overhead proportional to device count in Bamboo [59] and Mario [31] limit scalability over backhaul.

4 Spare Compute Characterization

The adoption of GPUs as accelerators in vRAN deployments raises a key question: *how much spare compute actually exists, and along which dimensions can it be safely harvested without compromising RAN performance?* We characterize this from two perspectives: (i) *Micro-scale*: fine-grained opportunities within each slot and within the GPU microarchitecture, capturing the multi-dimensional nature of GPU utilization (compute, bandwidth, and SM occupancy) and the fluctuating, hard-to-predict RAN load caused by traffic variation.

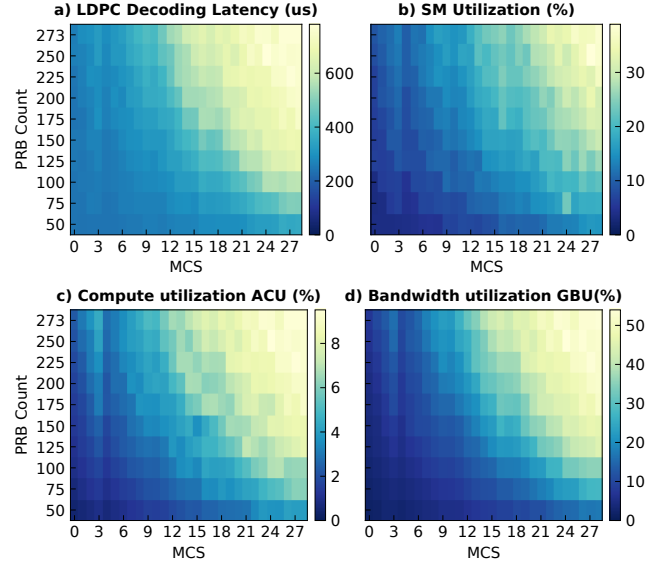


Figure 1: Max (worst case) latency (a), SMU (b), ACU (c), and GBU (d) of LDPC decoding under RAN contexts (#PRBs and MCS index).

(ii) *Macro-scale*: longer-term behavior across time at a single site and across multiple sites, examining whether spare compute is consistently available at the network level.

Answering this question requires more than coarse, scalar utilization metrics as in prior works (e.g., [51]). Also, for the micro-scale level, standard monitoring tools (e.g., `nvidia-smi`) only report whether *any* kernel is active within a ~ 1 s window and do not expose multi-dimensional GPU utilization metrics [38]. At the macro-scale level, traffic traces [45] capture millisecond-level traffic but do not directly translate into actual GPU utilization.

We note that in this work, we focus on offloading the RAN LDPC decoding workload to the GPU, as is commonly done in the literature (e.g., [50, 51]). Our methodology for spare compute characterization, however, generalizes to full L1 pipeline offloading [36], and we also expect our key take-aways to hold for that case as well.

4.1 Micro-Scale Opportunity

Our experimental platform builds on SionnaRK [10], integrating the OpenAirInterface (OAI) RAN stack with GPU-accelerated LDPC decoding on an NVIDIA DGX Spark (48 SMs, all available to the RAN). We profile decoding latency using the OAI `ulsim` tool [32], sweeping MCS index (0–28) and PRB count (0–273) across the full configuration space. The LDPC decoder is fixed at 10 iterations, capturing worst-case execution time and making runtime independent of SNR.

Slack within a slot. We observe two types of spare compute for single DU processing within a slot: (i) *the most demanding RAN configuration does not fully use the deadline budget*: under MCS 28, 273 PRBs, decoding finishes in $\sim 700 \mu\text{s}$, while

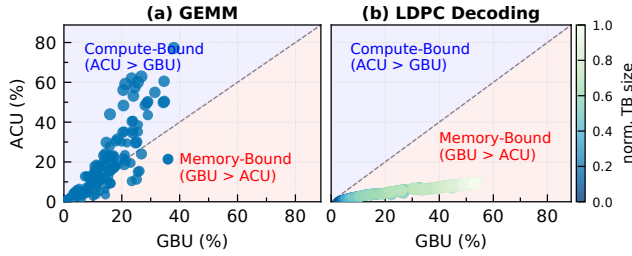


Figure 2: ACU and GBU of GPU operations for FM training (model sizes: 1.3B to 70B) vs. LDPC decoding.

others finish earlier, leaving at least 30–70% of every slot completely idle (Fig. 1a); (ii) *the most demanding RAN configuration does not fully use the GPU capacity*: worst-case SMU reaches only ~35%, and across the bulk of the operating region ($MCS \leq 20$, $PRB \leq 200$) it stays below 20%, meaning over 80% of SMs sit idle. LDPC decoding takes longer in time rather than stretching wider across SMs (Fig. 1b).

Microscopic (compute and memory) view of slack. Looking under the hood of SMU, we make two observations on spare compute after RAN use and implications for co-locating FM training: (i) *RAN is weak in compute intensity*: ACU even in the worst case (under MCS 28, 273 PRBs) reaches only ~12% (Fig. 1c). This is rooted in the latency goal of RAN design, leading to decoder kernels spreading thread blocks across many SMs to minimize latency rather than packing them onto fewer SMs for compute efficiency. (ii) *RAN exerts substantial pressure on memory*: GBU ranges from ~10% for small transport blocks (TBs) up to ~60% when code blocks are large and highly redundant (Fig. 1d). This makes LDPC decoding fundamentally bandwidth-bound, leaving compute pipeline mostly idle. We provide a side-by-side comparison of FM training with LDPC decoding in Fig. 2 across both ACU and GBU dimensions. *Interestingly, we observe that these workloads are structurally complementary in that training is compute-intensive, whereas RAN (LDPC decoding) is memory-bound.* It is also worth noting that FM inference (e.g., LLM decoding) has similar characteristics to those of RAN processing in being memory-bandwidth bound (Table 2).

Multi-DU still leaves compute on the table. We next profile the worst-case aggregate utilization for up to 8 DUs (matching common vRAN multiplexing settings [51]), with each DU driven at peak traffic (MCS 28, 273 PRBs) in every slot. From our observations, we see that the SMUs are 35.7/46.2/57.6/58.0% for 1/2/4/8 DUs. This leaves roughly 40% of spare compute capacity with 4 concurrent DUs. Notably, scaling up the number of DUs from 4 to 8 only yields a marginal increase in SMUs, as the memory bandwidth (GBU) and host CPU capacity to feed the GPU become the dominant bottleneck. In practice, the effective SMU is much lower for multi-DU settings, as the DU peaks will rarely overlap.

Table 2: GPU workload characteristics in terms of compute and memory-bandwidth demand (% of peak). Co-ex. marks coexistence compatibility with RAN.

Workload	Dominant Operation	Compute Demand	Bandwidth Demand	Co-ex.
RAN (LDPC Decoding)	Vector	0.3–3%	48–92%	—
FM training	Matrix	30–57%	moderate	✓
LLM decoding	Vector	<5%	60–90%	×

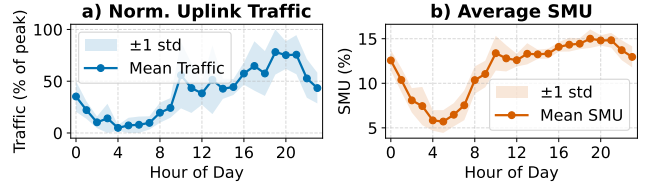


Figure 3: Hourly uplink traffic and corresponding average SMU across 24 hours for a single site.

4.2 Macro-Scale Opportunity

We replay per-slot uplink TB traffic captured by a large-scale production traffic trace [45], using the modified OAI PHY-TEST tool [32]. The trace [45] is collected from six production cell sites of three major operators in Madrid, covering urban and sub-urban zones. Here, we only present the average SMU, as the other two metrics exhibit similar patterns.

Single-site temporal spare-compute patterns. We investigate the hourly traffic variability and the corresponding SMU. We observe the following: (i) *Diurnal traffic pattern*: traffic remains very low with minimal variance during nighttime hours (02:00–06:00), while daytime exhibits several peaks (Fig. 3a). (ii) *SMU mirrors diurnal pattern with lower variance*: when translated to SMU, nighttime consistently yields a reliable multi-hour window where over 90% of SMs are idle and are available for non-RAN workloads; even during peak daytime hours, spare compute in terms of SMs remains stable around 85%, with notably lower variance than raw traffic volumes (Fig. 3b).

Spatial cross-site spare-compute patterns. We consider both absolute traffic and corresponding SMU across six cell sites. We observe: (i) *Strong volume skew across sites*: Zone-II sites (4–6) carry much higher load than Zone-I sites (1–3), which in turn implies that low-traffic sites have substantial spare compute even during their own local peaks (Fig. 4a); (ii) *Staggered daily peaks*: peaks occur at different hours—some sites (e.g., 1 and 3) peak around midday, whereas others (e.g., 4 and 5) peak in the late afternoon/evening—so sites are rarely all busy at once, and there are almost always one or two off-peak sites with spare SMs that can host additional work (Fig. 4b); (iii) *SMU mirrors traffic heterogeneity*: average SMU follows the same spatiotemporal pattern as traffic, reinforcing that spare compute is not only per-site and diurnal, but also complementary across sites (Fig. 4b).

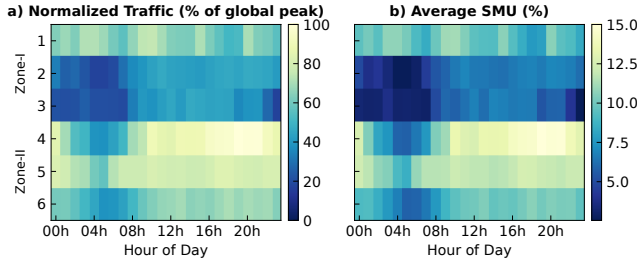


Figure 4: Spatial traffic patterns and their SMU.

4.3 Key Takeaways

Substantial spare compute exists at both micro and macro scales. Worst-case SMU stays below 35% within a slot (§4.1), and diurnal patterns leave over 90% of SMs idle during off-peak hours across sites (§4.2). Overall, we observe that as much as 40–85% spare compute capacity can be opportunistically harvested by non-RAN workloads, provided it can be safely exposed without hurting RAN performance. Furthermore, compute-intensive workloads such as FM training are better placed to coexist with memory bandwidth intensive RAN processing.

Spare compute is bursty at each site while complementary across cell sites. Per-slot compute demand fluctuates widely with MCS/PRB variation (§4.1), while cross-site peaks are staggered so not all sites experience peaks at the same time (§4.2). This in turn implies that spare compute is temporally fragmented at each site while spatially diverse across sites.

5 WEAVER

5.1 Goals and Challenges

In light of the observations from the characterization study from §4 on spare compute opportunities in GPU-accelerated RAN infrastructure, our system design presented in this section has **two goals**:

- 1) Design a mechanism that maximizes usable spare compute for non-RAN workloads while protecting RAN performance;
- 2) Considering FM training as a challenging and suitable non-RAN workload, devise an efficient training strategy that can best harness the available spare compute over the distributed AI-RAN compute infrastructure.

To achieve these goals, there are **two key challenges**:

C1: Stabilizing spare compute despite bursty RAN traffic. Although spare compute appears as headroom in all three dimensions – SMU, ACU and GBU (§4.1), those cannot be exploited directly: GPUs do not expose any mechanism to control these dimensions independently. The only runtime lever available is *SM count* [35] allocation, which jointly affects all three spare-compute dimensions (See Appendix A.1 for analysis). SM-level sharing must therefore be adopted to make spare compute both usable and controllable.

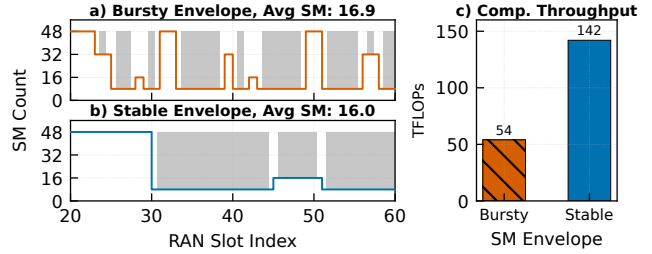


Figure 5: SM envelope comparison: a) bursty envelope derived directly from RAN traffic; b) stable envelope; c) impact of SM envelope on training computational throughput (the higher, the better).

To help elaborate this challenge, we define the notion of a **SM envelope**, that is, a time series of SM counts at the slot granularity that specifies how many SMs are reserved for RAN processing in each slot, and by complement, how many are exposed as spare compute to co-located workloads.

The inherent bursty nature of RAN traffic [45] when directly translated into the GPU usage results in an SM envelope with frequent SM transitions (Fig. 5a). Such an envelope, however, may not allow a non-RAN workload to exploit the available spare compute. For example, FM training throughput suffers with an SM envelope with fast time varying transitions (Fig. 5c), as each SM-count adjustment incurs at least non-trivial tens of microseconds of overhead. So, a stable envelope is preferred to make the spare compute usable (Fig. 5b). But stabilizing the SM envelope naively can hurt RAN performance by causing processing deadlines to be violated. Thus, we need a controlling mechanism that stabilizes the envelope while protecting RAN performance.

C2: Robust and efficient FM training strategy to cope with resource availability dynamics at different timescales within and across sites. This challenge arises from two issues, as outlined below: (i) *Residual step-level variation*: The stable SM envelope resulting from addressing C1 cannot fully provide a flat spare compute as having dedicated access to a standalone GPU. Instead, even after stabilizing SM usage, the exposed spare compute still changes at the granularity of several milliseconds – multiple PHY slots (Fig. 5b); (ii) *Conflicting control requirements*: micro-scale fluctuations require fast, lightweight reactions, while macro-scale heterogeneity across sites calls for infrequent but heavier restructuring of the training layout. A single flat controller to steer the training process cannot reconcile both, motivating a hierarchical approach that separates rapid slot-level responsiveness from slow adaptation across sites.

5.2 Design Overview

To meet the goals outlined above and address the associated challenges **C1** and **C2**, we present a system design termed

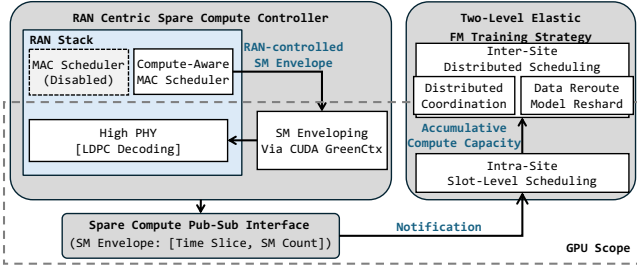


Figure 6: WEAVER system architecture overview.

“WEAVER”. WEAVER enables FM training workload to opportunistically and efficiently coexist with the primary RAN workload on GPU-accelerated RAN infrastructure while ensuring RAN performance is unaffected from infrastructure sharing. As illustrated in Fig. 6, WEAVER consists of two coupled components: (1) RAN-centric spare compute controller; and (2) Two-level elastic FM training strategy. A lightweight publish-subscribe (pub-sub) interface bridges the two: the RAN controller publishes a stable SM envelope; the co-located non-RAN workloads – FM training in this work – subscribe to it and adapt their inner workings to be continually in tune with the SM envelope. Below we outline the two core components of WEAVER.

(1) RAN centric spare compute controller (§6). Controlling GPU compute resource allocated to the RAN *externally* as done in previous work [50] necessitates prediction of future RAN SM demand, which is hard. Underestimation risks RAN performance while overestimation limits spare compute opportunity. Therefore, we design a controller that sits inside the RAN to modulate the spare compute for non-RAN workloads. This design allows prioritizing RAN as the first-class tenant and protect its performance, while stabilizing the SM envelope to maximize the utility of spare compute made available to other third-party workloads. The MAC scheduler is a natural control point: its PRB budgeting and scheduling decisions directly shape SM demand. We therefore design a *compute-aware MAC scheduler* that smooths RAN traffic spikes to stabilize the SM envelope across consecutive slots, while meeting per-slot RAN deadlines and maintaining UE performance. This compute-aware scheduler outputs its shaped SM envelope to the middleware interface.

(2) Two-level elastic FM training strategy (§7). We propose a tailored two-level FM training strategy that uses a hierarchical control structure to address the twin issues associated with C2. It relies on a centralized coordinator that sits at the natural aggregation point for the RAN infrastructure in the mobile core to track training progress, distribute computation across RAN sites and aggregate intermediate training state. At the *inter-site* level (§7.1), it invokes model-layout reconfiguration only when macro-scale spare-compute shifts

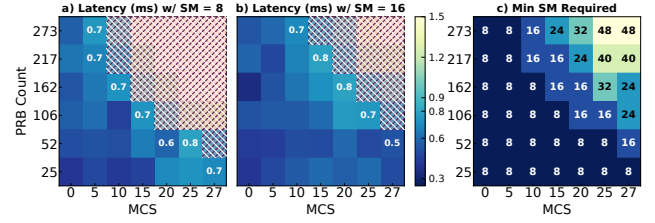


Figure 7: a) and b): Worst-case LDPC decoding latency with various (PRB, MCS) tuples given an SM count; c): minimum SM demand as a function of (PRB, MCS).

persist, otherwise uses lightweight rerouting and recomputation to absorb short-term fluctuations. At the *intra-site* level (§7.2), it uses a local adaptation loop to absorb slot-scale compute resource variation by sharding GEMM operations in real-time to match the instantaneous spare-compute budget.

6 RAN Centric Spare Compute Controller

As stated in §5, the RAN centric spare compute controller in WEAVER actively shapes RAN compute demand through a compute-aware MAC scheduler. Its objective is to keep the SM envelope static across consecutive slots while preserving RAN performance: every uplink grant must remain decodable within the slot deadline, and per-UE performance must not be degraded.

6.1 Core Idea

The idea behind compute-aware scheduling is that *shaping* RAN compute demand requires *rebalancing* PRB allocation across slots. Because uplink traffic is typically light and spiky, granting PRBs for every instantaneous spike would force the SM envelope to track these rapid swings. Instead, WEAVER smooths demand by withholding PRBs at low SM tiers and draining the buffer backlog in controlled batches once a threshold is reached. This suppresses slot-level volatility and produces a much more stable SM envelope, while still guaranteeing that all grants are served within their deadlines.

A key enabling property is that SM demand is directly controllable through PRB allocation. Given an MCS index, capping the PRBs limits the required SM count within a slot. This relationship is captured by an offline-generated lookup table (Fig. 7c). This table is derived from worst-case LDPC decoding latency profiling across SM budgets and RAN configurations (e.g., Fig. 7a–b), using *ulsim* as mentioned in §4.1. For example, (MCS 15, 162 PRB) meets the latency deadline with 16 SMs but violates it with 8 SMs (Fig. 7b); hence the table records 16 as the minimum (Fig. 7c). As a pre-deployment step, we calibrate this lookup table against the end-to-end RAN system to account for system-level effects not captured by *ulsim* (see more details in Appendix B.2).

Key issues to resolve. Compute-aware MAC scheduling must address the following two key issues:

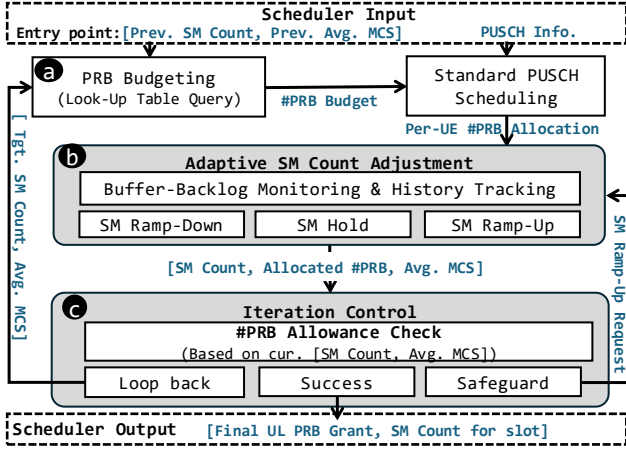


Figure 8: Per-slot compute-aware scheduling pipeline.

- **Determining PRB budget from SM count.** Deciding the total PRB budget from the lookup table requires both the SM count *and* the average MCS for the slot, but the true average MCS is known only *after* PRB allocation completes, creating a chicken-and-egg dependency—the scheduler must cap PRBs before allocation, yet average MCS is only finalized later.
- **Deciding when and how much to adjust the SM envelope.** Adjusting the SM envelope too frequently prevents co-located workloads from making forward progress, while adjusting too conservatively risks under-provisioning during bursts or over-provisioning during lulls. The scheduler must balance responsiveness to traffic spikes with stability across slots.

6.2 Key Mechanisms

The mechanisms in Fig. 8 jointly address both issues, forming an iterative per-slot compute-aware scheduling pipeline.

PRB budgeting. At the start of each slot and on each following re-entry iteration, the scheduler pairs an MCS estimation with the target SM count and queries the lookup table to obtain a provisional PRB budget. We define *average MCS* as the per-UE MCS weighted by their respective PRB allocations. The initial MCS estimate (average MCS) and the target SM count are carried from the previous slot’s scheduling decision; on re-entry, they are updated by the true average MCS and the adjusted SM count from the preceding iteration. Standard proportional-fair scheduling [42] then proceeds within the decided PRB budget to ensure UE fairness.

Adaptive SM count adjustment. Given UL PRB scheduling decisions, the scheduler inspects per-UE buffer backlogs as well as the history of SM envelopes:

- **Hold:** If all UE backlogs are empty or below a tolerance threshold, the SM count stays unchanged.
- **Ramp-up:** If any UE backlog exceeds the pre-defined threshold, the scheduler immediately increases the SM count to

drain the UE buffer backlog within the current slot, preserving UE performance.

- **Ramp-down:** If no ramp-up occurs, the scheduler checks whether the current slot could have been served with fewer SMs at a lower tier. A hysteresis counter tracks sustained over-provisioning and triggers a gradual step-down when appropriate. A “fast-exit” path applies when the current slot allocated zero PRBs, allowing prompt down-sizing without waiting for hysteresis, so that spare SMs can be released to co-located workloads.

This SM adjustment is asymmetric by design: ramp-up is immediate and unconditional to protect RAN performance, while ramp-down follows a two-branch path. Both hold, and slow-hysteresis ramp-down maximizes SM-envelope stability, while fast-exit ramp-down avoids unnecessary overprovisioning of GPU resources for RAN.

Iteration control with PRB allowance check. After PRB allocation and SM adjustment, both the true average MCS and the updated target SM count may deviate from the values assumed for PRB budgeting. To this end, the scheduler recomputes the PRB allowance (i.e., maximum supported PRB count) by querying the lookup table (Fig. 7c) with these two updated values. If the allocated PRBs are within a close margin *below* this allowance, the grants are directly emitted. Otherwise, the scheduler loops back to PRB budgeting with the updated average MCS and SM count. Then it starts another iteration repeatedly from steps (a)–(c) in Fig. 8, until PRB allocation converges, ultimately generating the current-slot UL PRB grant. A small iteration bound (i.e., five) safeguards against non-convergence; if exceeded, the scheduler forces an SM ramp-up to guarantee forward-progression.

By combining all three mechanisms above, our compute-aware scheduler determines the final UL PRB grant for each UE while exposing the corresponding SM count in the current slot. Over time, a stable SM envelope is produced, allocating only the minimum necessary SM count per slot. This maximizes the spare-compute opportunity available to the training workload while guaranteeing RAN performance. The algorithm details are seen in Appendix B.

7 Two-Level Elastic Training

Unlike datacenter training, where GPU capacity is effectively static once a job is placed, RAN sites exhibit *time-varying capacity* driven by spare-compute fluctuations (§4). We address this with two levels of scheduling hierarchy (shown in Fig. 9): (i) *inter-site scheduling*: harvest capacity across sites at slower timescales, from seconds to hourly traffic shifts; and (ii) *intra-site scheduling*: adapt to slot-level variation locally, and expose only an aggregated capacity view to the inter-site scheduler. This separation is essential because slot-level fluctuations (< 1 ms) are too fast for inter-site coordination (> 100 ms), while macro-scale shifts are too persistent (1 hour

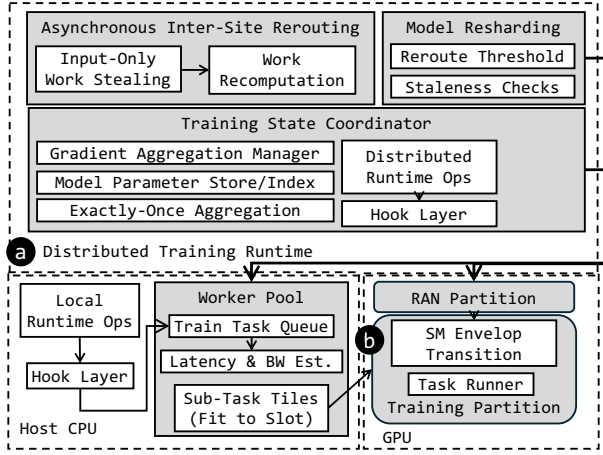


Figure 9: Two-level elastic training architecture: the inter-site coordinator manages AIR rerouting and MLR rebalancing across RAN sites (left), while each intra-site scheduler turns slot-level spare compute into stable execution units (right).

is 3.6M slots aggregated) to be handled efficiently by intra-site scheduling alone.

7.1 Inter-Site Scheduling

Inter-site scheduling must cope with model and optimizer state on the order of 10–100 GB at each site over 1–10 Gbps inter-site links, so migrating state would take seconds to minutes. Because RAN sites connect to the core in a star topology with 10–50× higher bandwidth on site-to-core links than indirect site-to-site paths, we use a coordinator-centric runtime that relies on these strong core links for lightweight control-plane coordination while minimizing data-plane state transfers between sites.

Centralized training state coordinator. The inter-site coordinator tracks where model state and in-flight work reside and enforces exact-once sample accounting as AIR reroutes micro-batches and MLR reshapes the layout. It is loosely coupled with the training runtime, hooking into existing synchronization points, such as forward checkpoints and collective communication, and observing micro-batch arrivals and completions at site boundaries for latency signals. In RAN deployments, the coordinator runs at the core to exploit strong site-to-core links, while in peer-to-peer clusters, the same logic can be hosted on a worker with replicated metadata storage. Detailed runtime metadata and protocol mechanics appear in Appendix C.1.

Asynchronous inter-site rerouting (AIR). The principle is *move work, not state* at sub-second timescales. AIR balances load across sites by rerouting micro-batches while keeping model state pinned at each site. At each control interval, the coordinator assigns new micro-batches to executors whose predicted completion time fits within the step budget. Micro-batches that exceed the estimated completion time

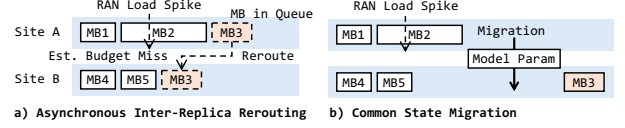


Figure 10: AIR scheduling illustration and benefits. MB refers to microbatch.

are deferred to another replica that holds the same model parameters, transferring only hidden states and recomputing gradients if necessary.

Consider the two-site example in Fig. 10 with two data-parallel replicas of the same work unit. If a RAN load spike slows site A while it executes MB2, AIR stops assigning new work to site A and reroutes the next queued micro-batch (MB3) to site B, while letting MB2 finish on site A. If MB2 is the last micro-batch in the step, AIR simply re-executes it on site B and commits whichever copy finishes first. Compared to migration-based approaches (Fig. 10b), AIR reacts without waiting for ongoing work to drain and moves only KB–MB-scale inputs or activations, not multi-GB model state.

Model layout rebalancing (MLR). AIR absorbs transient fluctuations by rerouting work within a fixed model-parallel layout. However, when one site remains persistently slower, deferrals accumulate, and the elastic horizon repeatedly overshoots across multiple control windows. In this case, the layout itself becomes the bottleneck, and rerouting alone cannot remove the structural imbalance. When the coordinator detects such sustained high deferral rates, the system escalates to MLR, a slow path that adjusts the model layout at barrier boundaries. Further discussion on supporting fault tolerance and rebalancing workflow is described in Appendix C, as well as a visual illustration.

7.2 Intra-Site Scheduling

Intra-site scheduling observes the per-slot SM envelope exported by the RAN controller and maintains a history window of accumulative capacity, so single-slot dips or spikes are smoothed before they reach the global scheduler. The result is a queue of short-lived, capacity-bounded slots that can be treated as a local worker pool, instead of exposing raw RAN traces with per-slot variability.

Slot-bounded compute units. Large GEMMs may exceed a slot window if spare compute shrinks mid-execution. WEAVER therefore decomposes GEMMs into sub-GEMM tiles that are admitted only when they satisfy two constraints under the current slot budget: (i) predicted completion before the slot deadline, and (ii) bandwidth fit within the remaining memory/interconnect headroom. WEAVER further uses persistent kernels [21, 44], where task queues are polled on the device in ~100 ns versus ~100 μs host launch latency, keeping decomposition compute-bound rather than launch-bound. This

Table 3: SM envelope stability and HARQ deadline compliance in two TRACTOR scenarios (4 UEs, 600 s).

Scen.	Method	Mean SM cnt.	SM trans.	ddl. met
Scen. 1	NoCTRL	14.2	435 (0.7/s)	99.8%
	YINYANGRAN	34.1	96 (0.16/s)	–
	WEAVER	14.4	89 (0.1/s)	99.8%
Scen. 2	NoCTRL	8.4	744 (1.2/s)	99.8%
	YINYANGRAN	22.5	124 (0.21/s)	–
	WEAVER	8.5	187 (0.3/s)	99.8%

allows transformer training kernels to be scheduled opportunistically at slot granularity. We detail how we solve other GPU-related overhead with pre-allocation in Appendix C.

8 Evaluation

8.1 RAN Centric Spare Compute Controller

Experiment Setup. We evaluate the RAN-control component of WEAVER on a full OAI [42] 5G NR stack using both trace-driven simulation (with OAI RFSIM) and Over-The-Air (OTA) validation (with a USRP B210 board and Android phones). The gNB is configured on band n78 (3.5 GHz TDD, 40 MHz bandwidth) deployed on a DGX Spark node [10].

For trace-driven simulation, two sets of traffic traces are used: (i) *TRACTOR 5G traces* [20]: these traces capture production O-RAN traffic from commodity 5G UEs, and two scenarios (Scenario 1: all UEs static with steady traffic; Scenario 2: UEs walking with dynamic traffic) are used. (ii) *D-ITG* [8]: synthetic uplink bursty UDP traffic traces are generated by this tool, capturing more extreme traffic conditions than those in (i). We further incorporate real-world channel dynamics by replaying uplink channel measurements from [20] alongside both traffic trace sets. This enables evaluation under realistic time-varying channel conditions, replacing the idealized static AWGN channel in RFSIM. More setup details are seen in Appendix D.1.

For OTA validation, we test two applications with different traffic profiles: (i) *iperf3* for sustained throughput and (ii) *WebRTC* video calls [5] for real-time interactive multimedia.

Baseline. WEAVER RAN-centric spare compute controller is the first to actively shape RAN compute demand for SM-envelope stability. We compare against two baselines: (i) NoCTRL, which derives the SM envelope directly from unmodified per-slot OAI MAC scheduling outputs; (ii) YINYANGRAN [50], representing the MPS-based GPU partitioning approach (see §2). To best evaluate its performance, we grant this baseline oracle knowledge: SM shares are repartitioned every second to exactly fit the ground-truth future RAN workload. RAN-side MPS is not enforced at runtime, as YINYANGRAN falls back to CPU during MPS reconfig ($\sim 0.3s$), which is impractical under the CPU resource constraints.

WEAVER creates stable SM envelopes with RAN deadline guarantee. Using TRACTOR’s realistic O-RAN traces,

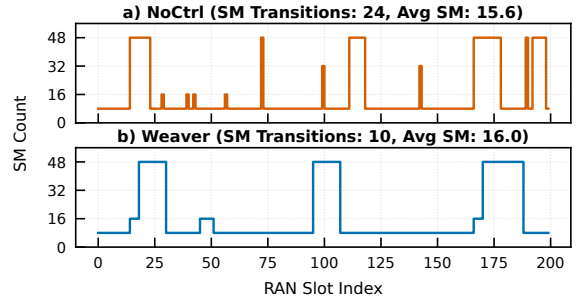


Figure 11: SM count timeline over a 200-slot window.
Table 4: Aggregated application-layer KPIs (3 UEs, 90 s UL UDP via D-ITG w/ real-world channel replay).

Metric	WEAVER	NoCtrl	Δ (%)
Throughput, sum (Mbps)	4.56	4.56	+0.1
Delay, mean (ms)	18.4	19.0	−3.1
Delay, P95 (ms)	120.2	124.3	−3.3
Jitter, mean (ms)	4.4	4.5	−3.0
Jitter, P95 (ms)	35.9	38.1	−5.9
Packet loss (%)	66.7	66.7	−0.03

Table 3 compares methods over a 600 s trace. NoCTRL has low mean SM count but frequent transitions; YINYANGRAN matches WEAVER in transitions but overprovisions SMs by 2.4–2.7 $\times$ due to fixed peak-sized MPS partitions for each second. WEAVER reduces transitions by 4.4–4.9 $\times$ over both baselines while maintaining mean SM counts close to NoCTRL and matching its 99.8% HARQ deadline-met rate (elaborated in Appendix D.1). *The produced SM envelope is exposed for training evaluation (§8.2).* These results hold under realistic channel dynamics (SINR: 2.3–20.7 dB, thousands of MCS transitions), effectively absorbed by the compute-aware scheduler’s PRB budgeting and iteration control (§6.2). As shown in Fig. 11, NoCTRL exhibits sharp SM spikes under bursty traffic, while WEAVER produces wide, stable plateaus.

WEAVER does not degrade RAN user performance. We compare application-level KPIs between WEAVER and NoCTRL under stress-test conditions (D-ITG bursty uplink traces saturating the link at ~ 13.2 Mbps for three co-served UEs). As shown in Table 4, throughput and packet loss are indistinguishable between the two configurations, while delay and jitter show slight improvements. Table 5 further confirms consistent results over-the-air with two schemes running in the same RF environment, where all deltas remain within $\pm 6\%$ and mixed in sign, consistent with natural OTA variation (stemming from UE background activity and uncontrolled RF interference etc.). Together, these results validate that WEAVER does not negatively impact user performance under either RFSIM or OTA conditions.

8.2 Training System on Cellular Testbed

Testbed. We run training on a GPU-backed 5G testbed with 8 $\times$ NVIDIA L40S with 48GB memory, each separated across

Table 5: OTA results (2 UEs): iperf (5 min bidirectional); WebRTC call quality (10 min bidirectional).

Test	Metric	WEAVER	NoCtrl	Δ (%)
iperf	Throughput (Mbps)	12.1	12.2	-0.8
WebRTC	Avg. bitrate (kbps)	2440	2460	-0.8
	Packet loss (%)	0.35	0.33	+6.1
	Jitter (ms)	3.4	3.3	+3.0
	RTT (ms)	22.4	22.8	-1.8
	Freeze count	0	1	—

different cities (Appendix D.2 shows the inter-site bandwidth and latency). We use the SM envelope trace from §8.1 to reflect millisecond-level dynamics, while rescaling the total number of SMs to the current GPU capacity.

Baselines. For the training side, we compare: (i) DTFM [67], using its DP+PP deployment and coarse-grained checkpoint-and-restore for handling worker dynamics; (ii) ASTEROID [66] with DP+PP uses a lightweight pipeline replay mechanism to recover from stragglers; and (iii) CONFIDANT [11] with PP, whose hybrid strategy combines local on-device updates with periodic global aggregation to tolerate slow or temporarily offline devices. The baseline GPU sharing method reuses the one from §8.1.

Models and Datasets. We use OPT [70] models as representative foundation models. Architecturally, OPT follows the same transformer design as LLaMA [60] and GPT [9], while spanning a wide range of scales (125M-70B), enabling scalability evaluation. We train OPT on SST-2 [56] with batch size 32 and sequence length 512 as a finetuning case defined in the OPT paper. The micro-batch size is determined by the training scheduler based on the chosen data-parallel (DP) and pipeline-parallel (PP) configuration.

WEAVER achieves substantial end-to-end training gains. We compare WEAVER against baselines that use MPS together with their own training systems. All methods are evaluated under the same trace replay over 1000 training steps, covering both per-site capacity fluctuations and cross-site drift. We report results using the full WEAVER stack end-to-end. Fig. 12 summarizes the results. WEAVER achieves 2.1 $\times$ higher throughput (35K tokens/s) than the best baseline (CONFIDANT, 17K tokens/s) on OPT-1.3B, with the gap widening to 4.2 $\times$ at OPT-6.7B (6.3K vs. 1.7K tokens/s). The gap among baselines is from model parallelism, with heavy DP AllReduce stalling training on communication, while CONFIDANT prefers PP and WEAVER adopts the same, thus performing better than others. Further gains come from both compute harvesting and dynamics adaptation. Especially, the gap grows with model size because larger models amplify the fixed cost of coarse-grained reconfiguration: DTFM’s checkpoint-and-restore and ASTEROID’s pipeline replay both consume a larger fraction of the per-step compute budget.

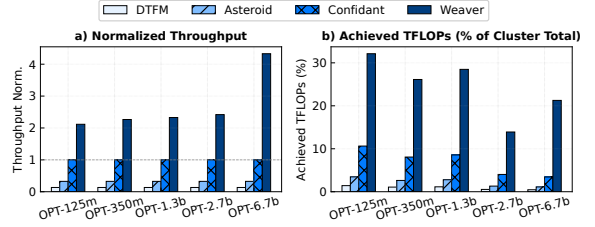


Figure 12: Throughput and achieved TFLOPS% under different OPT model sizes. Practical maximum TFLOPS% ranges from 23%–55% varied with network topology and model type [26, 55].

WEAVER better harvests compute on RAN sites. We isolate the contribution of intra-site scheduling: (i) we select a 2 min trace segment in which the SM envelope exhibits only local fluctuations, without any macro-level shift; and (ii) we disable all adaptive cross-site scheduling for all methods. Under this isolation, WEAVER harvests 83% of available spare compute while maintaining only 0.6% RAN deadline violations, versus 32% harvesting for YINYANGRAN (more details in Appendix D.4). This explains the base 1.3 $\times$ improvements from baseline in token throughput. The key difference is temporal resolution: WEAVER adapts at sub-slot granularity (< 1 ms), while YINYANGRAN’s 1-second MPS loose capping leaves 75% of transient spare compute unharvested.

WEAVER better adapts to dynamic RAN capacity. We isolate the contribution of inter-site scheduling as follows: (i) we enable our single-site scheduling plugin for all baselines, ensuring identical intra-site behavior; and (ii) we initialize all methods with the same model-parallel partition. Over a 10-minute trace with 41 capacity-drop events across 8 sites, WEAVER achieves 15–32% of peak cluster FLOPS, compared to 6–11% for CONFIDANT and 1–2% for DTFM. The gap stems from two sources: (i) *MPS reconfiguration vs. SM envelopes*: under traffic fluctuations, baselines incur 374–453 ms of MPS reconfiguration per adjustment. As a result, GPUs spend about 40% of the runtime idle waiting for configuration changes rather than executing work, which alone accounts for the observed 1.4 $\times$ performance gain. (ii) *Heavy state migration vs. lightweight data rerouting*: baselines also stall during capacity-shift events, which occur on average twice per minute, taking up to 2.7 s for checkpoint-and-restore or up to 8.5 s for global rebalancing via layer migration, marking up the final source of gain. In contrast, WEAVER adapts using elastic rerouting with a median overhead of 247 ms (including 67 ms network delay), resulting in only 13.7% of wall-clock training time lost to recovery.

8.3 Large Scale Evaluation of Training

Environment settings. We evaluate under three environment settings: (i) *Rescaled*: replicating real testbed topology

Table 6: Device-scaling throughput and achieved cluster FLOPS across three environments for OPT-13B.

Devices	System	Rescaled		Typical		Ideal	
		tok/s	FLOPS%	tok/s	FLOPS%	tok/s	FLOPS%
128	WEAVER	76.8K	12.6%	106.7K	17.7%	121.0K	56.1%
	Confidant*	47.6K	7.8%	58.5K	8.6%	107.7K	30.2%
	Confidant	46.6K	7.8%	46.5K	7.8%	101.3K	30.5%
512	WEAVER	92.2K	10.8%	350.6K	12.7%	484.8K	51.8%
	Confidant*	47.6K	2.3%	87.3K	3.8%	182.8K	12.3%
	Confidant	46.6K	2.3%	49.7K	1.7%	101.3K	9.8%

and interconnect to a larger number of nodes while preserving the measured site and link characteristics; (ii) *Typical*: modeling a representative RAN deployment with a star topology, where each RAN site connects to the core at 25 Gbps with 1 ms latency, core-to-core links provide 400 Gbps with 20 ms latency, and direct peer-to-peer links between sites are limited to 10 Gbps, with 32 sites per core; and (iii) *Ideal*: using unlimited bandwidth and zero latency, together with the largest currently available GPU memory capacity of 144 GB (e.g., NVIDIA GH200-based Aerial testbeds [39]).

Methodology. For single-node performance, we run training with a reduced number of layers, or isolate the dominant computational kernels within each layer, while keeping batch size and sequence length unchanged, and profile the resulting end-to-end latency. For communication, we transmit the actual hidden-state tensors required by the training framework and measure the achieved network performance directly. Using these compute and communication profiles, we extrapolate beyond what is feasible to run end-to-end on our physical testbed, covering models from OPT-13B to OPT-70B and cluster sizes from 128 to 512 devices. We introduce a stronger variant, CONFIDANT*, which exhaustively searches all DP+PP parallelism combinations to select the best layout and assumes perfect per-slot compute harvesting. Full results are shown in Appendix D.3.

WEAVER better matches RAN network topology. Centralized aggregation better exploits the high-bandwidth core network and is therefore faster, with 2–7 $\times$ higher throughput. CONFIDANT* P2P communication is either bottlenecked by slower edge links (*Rescaled*) or congests the core when many workers communicate simultaneously (*Typical*); its own improvement is due to an increase in total backhaul capacity. In contrast, WEAVER simply delivers data through the backhaul link, and each link can work independently. At 512 devices, the gap widens: CONFIDANT*'s communication overhead consumes 73% of step time as each site still sends a fixed amount of data versus WEAVER scaling linearly.

WEAVER scales to more sites. For baselines, DP+PP alone cannot scale further than 128 sites due to micro-batch and

Table 7: Model-scaling throughput on 512 devices.

Model	System	Peak Mem (GB)	Throughput (tok/s)		
			Rescaled	Typical	Ideal
OPT-13B	WEAVER	13.8	92.2K	350.6K	484.8K
	Confidant*	13.9	47.6K	87.3K	182.8K
	Confidant	27.9	46.6K	49.7K	101.3K
OPT-70B	WEAVER	40.2	15.4K	58.4K	80.8K
	Confidant*	75.2	OOM	OOM	22.9K
	Confidant	149.8	OOM	OOM	OOM

layer partition limitations. WEAVER composes tensor parallelism and finer-grained sharding to expose additional parallelism and sustain scalability as sites increase (Table 12). WEAVER maintains 87% scaling efficiency at 512 devices under *Typical* (compared to 27% for CONFIDANT*), with communication overhead of only 13% versus 63% for baselines. Under *Ideal*, which removes all network bottlenecks, WEAVER's advantage narrows but persists (4.8 $\times$ vs 7.1 $\times$ under *Typical*), confirming that the gains are not purely network-dependent. CONFIDANT series throughput is effectively flat across device counts under *Rescaled* and *Typical* (46.5–49.7K tok/s from 128 to 512), as DP+PP parallelism exhausts its configuration space and additional sites sit idle. Even CONFIDANT*, which searches all parallelism layouts, scales only from 58.5K to 87.3K tok/s (1.5 $\times$ on 4 $\times$ hardware) under *Typical*.

WEAVER scales to larger models. For large models such as OPT-70B, naive placement can run out of GPU memory. WEAVER complements pipeline parallelism with finer-grained GEMM-level sharding so that model state is distributed more evenly, avoiding OOM while preserving training progress (Table 13). Under *Rescaled* and *Typical* (48 GB GPUs), CONFIDANT and CONFIDANT* run out of memory starting at OPT-30B, while WEAVER's sharding splits *within* layers, making it the only system capable of training OPT-30B and OPT-70B under realistic RAN conditions. Under *Ideal* CONFIDANT* fits OPT-30B and OPT-70B with its additional DP in CONFIDANT* helps the scaling, but WEAVER still achieves 3.1 $\times$ higher throughput at OPT-30B (202.0K vs 65.3K tok/s) and 3.5 $\times$ at OPT-70B (80.8K vs 22.9K tok/s)—confirming that the advantage extends beyond memory efficiency to scheduling and parallelism quality.

9 Conclusions

This paper presents WEAVER, a system that enables FM training to safely and efficiently harvest spare GPU compute from AI-RAN compute infrastructure. By co-designing a RAN-centric spare compute controller with a two-level elastic FM training strategy, WEAVER transforms bursty spare capacity into stable SM envelopes and enables robust training that efficiently adapts to resource variability across sites and timescales. Our evaluation shows that WEAVER significantly improves FM training throughput while guaranteeing RAN performance.

References

- [1] 3GPP. NR; Medium Access Control (MAC) protocol specification. Technical Specification (TS) 38.321, 3rd Generation Partnership Project (3GPP), 2024.
- [2] AI-RAN Alliance. AI-RAN alliance vision and mission white paper, 2024.
- [3] D. P. Anderson. BOINC: A platform for volunteer computing. *Journal of Grid Computing*, 18(1):99–122, 2020.
- [4] A. L. Beberg, D. L. Ensign, G. Jayachandran, S. Khaliq, and V. S. Pande. Folding@home: Lessons from eight years of volunteer distributed computing. In *IPDPS*, pages 1–8. IEEE, 2009.
- [5] N. Blum, S. Lachapelle, and H. Alvestrand. WebRTC: real-time communication for the open web platform. *Communications of the ACM*, 64(8):50–54, 2021.
- [6] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. v. Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, et al. On the opportunities and risks of foundation models, 2021.
- [7] L. Bonati, M. Polese, S. D’Oro, S. Basagni, and T. Melodia. Open, programmable, and virtualized 5G networks: State-of-the-art and the road ahead. *Computer Networks*, 182:107516, 2020.
- [8] A. Botta, A. Dainotti, and A. Pescapé. A tool for the generation of realistic network workload for emerging networking scenarios. *Computer Networks*, 56(15):3531–3547, 2012.
- [9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *NeurIPS*, 2020.
- [10] S. Cammerer, G. Marcus, T. Zirr, F. A. Aoudia, L. Maggi, J. Hoydis, and A. Keller. Sionna research kit: A GPU-accelerated research platform for AI-RAN. 2025.
- [11] Y. Chen, Y. Yan, S. Ge, Y. Qin, Y. Zheng, Q. Yang, S. He, Z. Shi, J. Chen, and Y. Shu. Confidant: Customizing transformer-based LLMs via collaborative training on mobile devices. In *ACM MobiCom*, pages 483–497. ACM, 2025.
- [12] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- [13] P. H. Coppock, B. Zhang, E. H. Solomon, V. Kypriotis, L. Yang, B. Sharma, D. Schatzberg, T. C. Mowry, and D. Skarlatos. LithOS: An operating system for efficient machine learning on GPUs. In *SOSP*, pages 1–17. ACM, 2025.
- [14] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia. Parcae: Proactive, liveput-optimized DNN training on preemptible instances. In *NSDI*, pages 1121–1139. USENIX Association, 2024.
- [15] A. Duttagupta, M. Jabbari, C. Fiandrino, M. Fiore, and J. Widmer. SYMBXRL: Symbolic explainable deep reinforcement learning for mobile networks. In *IEEE INFOCOM*, pages 1–10. IEEE, 2025.
- [16] L. Emberson and D. Owen. The stock of computing power from NVIDIA chips is doubling every 10 months, 2025.
- [17] R. Fan, T. Ren, M. Xie, S. Gao, J. Shu, and Y. Lu. GPREENPT: GPU preemptive scheduling made general and efficient. In *ATC*, 2025.
- [18] X. Foukas and B. Radunovic. Concordia: teaching the 5G vRAN to share compute. In *ACM SIGCOMM*, pages 580–596. ACM, 2021.
- [19] C. Ge, A. Mahimkar, Z. Ge, R. Fernandez, J. Maniaci, S. Pathak, and M. Shah. Iridescence: Improving configuration tuning in the presence of confounders for 5G NSA networks. *Proc. ACM Netw.*, 3(CoNEXT1):1–22, Mar. 2025.
- [20] J. Groen, M. Belgiovine, U. Demir, B. Kim, and K. Chowdhury. TRAC-TOR: Traffic analysis and classification tool for open RAN. In *IEEE ICC*, pages 4894–4899. IEEE, 2024.
- [21] K. Gupta, J. A. Stuart, and J. D. Owens. A study of persistent threads style GPU programming for GPGPU workloads. In *InPar*, pages 1–14. IEEE, 2012.
- [22] L. Han, Z. Zhou, and Z. Li. Pantheon: Preemptible multi-DNN inference on mobile edge GPUs. In *MobiSys*, pages 465–478. ACM, 2024.
- [23] M. Han, H. Zhang, R. Chen, and H. Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *OSDI*, pages 539–558, 2022.
- [24] S. Hui, H. Wang, Z. Wang, X. Yang, Z. Liu, D. Jin, and Y. Li. Understanding mobile traffic patterns of large scale cellular networks: The shanghai cellular network dataset. *Proc. ACM IMWUT*, 18(4), 2024.
- [25] Y. Jia, Y. Zhong, M. Wang, J. Gao, P. Zhang, X. Liu, and X. Jin. Aquifer: Transparent microsecond-scale scheduling for vRAN workloads. *IEEE Transactions on Services Computing*, 17(6):3171–3184, 2024.
- [26] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu. MegaScale: Scaling large language model training to more than 10, 000 GPUs. In *NSDI*, pages 745–760. USENIX Association, 2024.
- [27] S. Jin, S. Kim, S. Ha, and K. Lee. End-to-end coordination of RAN and edge server for latency-critical inference serving over cellular networks. *Proc. ACM Netw.*, 3(CoNEXT4):1–23, Nov. 2025.
- [28] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.
- [29] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. 2020.
- [30] L. Kundu, X. Lin, R. Gadiyar, J.-F. Lacasse, and S. Chowdhury. AI-RAN: Transforming RAN with AI-driven computing infrastructure. *IEEE Communications Magazine*, 64(1):168–174, 2026.
- [31] W. Liu, M. Li, G. Tan, and W. Jia. Mario: Near zero-cost activation checkpointing in pipeline parallelism. In *PPoPP*, pages 197–211. ACM, 2025.
- [32] F. Mani. Introduction to 5G RAN PHY simulators in OpenAirInterface. OAI Webinar Series, Chapter 4, 2025.
- [33] Nokia. Nokia partners with NVIDIA, 2025.
- [34] NVIDIA. NVIDIA AI Aerial.
- [35] NVIDIA. *CUDA C++ Programming Guide*, 2024. Version 12.6.
- [36] NVIDIA. NVIDIA Aerial SDK: GPU-accelerated 5G radio access network, 2024.
- [37] NVIDIA. NVIDIA cuPHY: GPU-accelerated 5G physical layer, 2024.
- [38] NVIDIA. *NVIDIA Data Center GPU Manager (DCGM)*, 2024. <https://developer.nvidia.com/dcgmn>.
- [39] NVIDIA. Enhanced DU performance and workload consolidation for 5G/6G with NVIDIA aerial CUDA-accelerated RAN, 2025.
- [40] A. F. Ocampo, M.-R. Fida, J. F. Botero, A. Elmokashfi, and H. Bryhni. Opportunistic CPU sharing in mobile edge computing deploying the cloud-RAN. *IEEE Transactions on Network and Service Management*, 20(3):2201–2217, 2023.
- [41] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, et al. GPT-4 technical report, 2023.
- [42] OpenAirInterface. OpenAirInterface: 5G software alliance, 2024.
- [43] OpenCelliD. OpenCelliD – the world’s largest open database of cell towers, 2025.

- [44] M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *PPoPP*, pages 429–431. ACM, 2023.
- [45] P. F. Pérez, C. Fiandrino, and J. Widmer. Characterizing and modeling mobile networks user traffic at millisecond level. In *WiNTECH*, pages 64–71. ACM, 2023.
- [46] Z. Qi, Y. Yao, Y. Li, C.-H. Tung, J. Zheng, D. Zhuo, and T. Chen. Decodex: Exploring and benchmarking of LDPC decoding across CPU, GPU, and ASIC platforms, 2025.
- [47] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, pages 10674–10685. IEEE, 2022.
- [48] M. Ryabinin, T. Dettmers, M. Diskin, and A. Borzunov. Swarm parallelism: Training large models can be surprisingly communication-efficient. In *ICML*, 2023.
- [49] J. X. Salvat Lozano, J. A. Ayala-Romero, A. Garcia-Saavedra, and X. Costa-Perez. Kairos: Energy-efficient radio unit control for O-RAN via advanced sleep modes. In *IEEE INFOCOM*, pages 1–10. IEEE, 2025.
- [50] L. L. Schiavo, J. A. Ayala-Romero, A. Garcia-Saavedra, M. Fiore, and X. Costa-Perez. YinYangRAN: Resource multiplexing in GPU-accelerated virtualized RANs. In *IEEE INFOCOM*, pages 721–730. IEEE, 2024.
- [51] L. L. Schiavo, G. Garcia-Aviles, A. Garcia-Saavedra, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Perez. CloudRIC: Open radio access network (O-RAN) virtualization with shared heterogeneous computing. In *ACM MobiCom*, pages 558–572. ACM, 2024.
- [52] R. Shafin, L. Liu, V. Chandrasekhar, H. Chen, J. Reed, and J. C. Zhang. Artificial intelligence-enabled cellular networks: A critical path to beyond-5G and 6G. *IEEE Wireless Communications*, 27(2):212–217, 2020.
- [53] S. D. A. Shah, M. Hafeez, A. Salama, and S. A. R. Zaidi. Proactive AI-and-RAN workload orchestration in O-RAN architectures for 6G networks. *IEEE Open Journal of the Communications Society*, 6:7939–7954, 2025.
- [54] W. Shen, M. Han, J. Liu, R. Chen, and H. Chen. XSched: Preemptive scheduling for diverse XPU. In *OSDI*, 2025.
- [55] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. 2019.
- [56] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, pages 1631–1642. ACL, 2013.
- [57] F. Strati, X. Ma, and A. Klimovic. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *EuroSys*, pages 1075–1092. ACM, 2024.
- [58] C. Sun, U. Pawar, M. Khoja, X. Foukas, M. K. Marina, and B. Radunovic. SpotLight: Accurate, explainable and efficient anomaly detection for open RAN. In *ACM MobiCom*, pages 923–937. ACM, 2024.
- [59] J. Thorpe, P. Zhao, J. Eyolfson, Y. Qiao, Z. Jia, M. Zhang, R. Netravali, and G. H. Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *NSDI*, 2023.
- [60] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, et al. Llama 2: Open foundation and fine-tuned chat models. 2023.
- [61] V. Volkov. *Understanding Latency Hiding on GPUs*. PhD thesis, EECS Department, University of California, Berkeley, Aug. 2016.
- [62] Z. Wu, R. Doost-Mohammady, and A. Sabharwal. ETHOS: Demystifying performance, energy, and computational efficiency in virtualized 5G O-RAN networks. In *WiNTECH*, pages 97–104, 2025.
- [63] D. Xu, A. Zhou, G. Wang, H. Zhang, X. Li, J. Pei, and H. Ma. Tutti: coupling 5G RAN and mobile edge computing for latency-critical video analytics. In *ACM MobiCom*, pages 729–742. ACM, 2022.
- [64] F. Xu, Y. Li, H. Wang, P. Zhang, and D. Jin. Understanding mobile traffic patterns of large scale cellular towers in urban environment. *IEEE/ACM Transactions on Networking*, 25(2):1147–1161, 2017.
- [65] K. Xu, R. Singh, M. Fiore, M. K. Marina, H. Bilen, M. Usama, H. Benn, and C. Ziemlicki. SpectraGAN: spectrum based generation of city scale spatiotemporal mobile network traffic data. In *CoNEXT*, pages 243–258. ACM, 2021.
- [66] S. Ye, L. Zeng, X. Chu, G. Xing, and X. Chen. Asteroid: Resource-efficient hybrid pipeline parallelism for collaborative DNN training on heterogeneous edge devices. In *ACM MobiCom*, pages 312–326. ACM, 2024.
- [67] B. Yuan, Y. He, J. Q. Davis, T. Zhang, T. Dao, B. Chen, P. Liang, C. Re, and C. Zhang. Decentralized training of foundation models in heterogeneous environments. In *NeurIPS*, 2022.
- [68] T. Zanouda, M. Masoudi, F. G. Gebre, and M. Dohler. Telecom foundation models: Applications, challenges, and future trends, 2024.
- [69] S. Zhang, L. Diao, C. Wu, Z. Cao, S. Wang, and W. Lin. Hap: SPMD DNN training on heterogeneous GPU clusters with automated program synthesis. In *EuroSys*, pages 524–541. ACM, 2024.
- [70] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, et al. Opt: Open pre-trained transformer language models. 2022.
- [71] S. Zhang, A. Xu, Q. Chen, H. Zhao, W. Cui, Z. Wang, Y. Li, L. Xiao, and M. Guo. Efficient performance-aware GPU sharing with compatibility and isolation through kernel space interception. In *ATC*, 2025.
- [72] Y. Zhang, I. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini. Faster and cheaper serverless computing on harvested resources. In *SOSP*, pages 724–739. ACM, 2021.

A Spare Compute Characterization

A.1 Effect of SM Enveloping on GPU Utilization

Fig. 13 compares per-slot utilization on a 48-SM GPU (MCS 15, 162 PRBs) with and without SM enveloping. The main takeaway is that enveloping trades a modest latency increase for substantially more stable and shareable headroom.

SM Enveloping creates headroom in all three dimensions. Without enveloping (Fig. 13a), LDPC decoding bursts across all 48 SMs for $\sim 550 \mu\text{s}$ (SMU $\sim 100\%$, GBU $\sim 65\%$, ACU $\sim 40\%$), after which the GPU sits idle for the remaining $\sim 450 \mu\text{s}$. With an envelope of 16 SMs (Fig. 13b), the same work is spread over a longer interval: SMU stabilizes at $\sim 33\%$ for $\sim 820 \mu\text{s}$, and the freed resources along all three dimensions become available for spatial sharing with co-located workloads.

GBU drops because fewer SMs issue concurrent memory requests, spreading the same traffic over a longer window. ACU also drops because fewer SMs are active, but in our measurements the per-SM ACU within the active SM *increases*: denser thread-block packing gives the warp scheduler more warps to interleave compute with memory stalls [35, 61].

3 $\times$ fewer SMs incur only 1.5 $\times$ slowdown. This headroom comes at a relatively small cost in completion time. Reducing the SM allocation by 3 $\times$ (from 48 to 16 SMs) increases decoding latency by only 1.5 $\times$ ($\sim 550 \mu\text{s}$ to $\sim 820 \mu\text{s}$), because fewer SMs each host more resident warps, improving per-SM occupancy and hiding memory stalls more effectively.

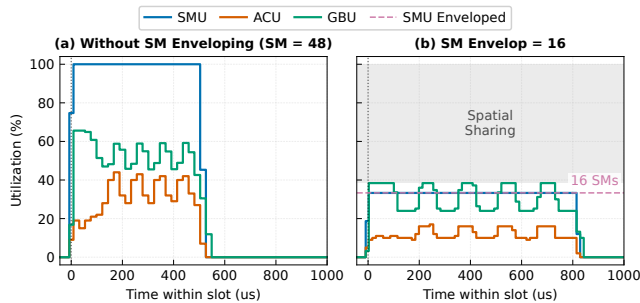


Figure 13: Per-slot GPU utilization (SMU, ACU, GBU) without and with SM Enveloping: the gray region shows headroom for spatial sharing.

B Compute-Aware Scheduler Details

This appendix elaborates on §6. It first presents the full pseudocode of the compute-aware UL scheduler, then explains the lookup table that maps RAN configurations to SM demand, and finally summarizes the parameters that govern its responsiveness and stability.

B.1 Pseudocode

Algorithm 1 instantiates the per-slot control loop described in Fig. 8. It takes the standard PUSCH scheduler input together with persistent SM state, then wraps the OAI proportional-fair UL scheduler with three scheduler-specific mechanisms: PRB budgeting, adaptive SM adjustment, and iteration control. The output is a set of UL PRB grants together with the SM budget committed for the next slot.

The control flow consists of four stages: (a) query the lookup table to obtain a provisional PRB budget, (a') run standard PUSCH scheduling within that budget, (b) adjust the target SM count based on observed backlog and recent history, and (c) recompute the PRB allowance under the updated state, looping whenever the current allocation is either above that allowance or more than a small tolerance below it.

B.2 Lookup Table

The lookup table captures the relationship between RAN configuration and SM demand. It is indexed by (MCS, PRB count, SM count) and records whether LDPC decoding meets the slot deadline for each triple. During scheduling, the controller uses it in two directions: PRB budgeting asks how many PRBs a given SM budget can support, while SM adjustment asks how many SMs are required for a given PRB demand. These two accesses are exposed as LUT_PRB and LUT_SM, which are different projections of the same table:

- `LUT_PRB(mcs, sm_count)`: fixes the MCS index and SM count, then returns the maximum PRB count whose decoding meets the slot deadline.
- `LUT_SM(mcs, num_prbs)`: fixes the MCS index and PRB count, then returns the minimum SM count that meets the slot deadline.

We construct the table in two steps. First, we profile standalone LDPC decoding with the OAI `ulsim` tool: for each (MCS, PRB, SM count) triple, decoding is benchmarked on the target GPU with SM enveloping applied, and the P99 latency is checked against a conservative $800 \mu\text{s}$ slot decoding budget. Second, we calibrate the table against end-to-end OAI runs with `rfsim` under synthetic extreme channel conditions and workloads, ensuring that the resulting budgets introduce no additional HARQ deadline violations beyond the baseline scheduler. For (MCS, PRB) pairs between profiled sample points, we apply ceiling interpolation in both dimensions so every query remains conservative by construction.

B.3 Parameters

Table 8 lists the tunable parameters used by the scheduler. These parameters control four things: the SM quantization granularity, how aggressively the scheduler ramps up in

Algorithm 1 Compute-Aware UL Scheduler: per-slot pipeline

Require: Scheduler input for standard PUSCH scheduling

Require: Persistent SM history state (carried across slots):

sm_prev : prev-slot SM count; mcs_prev : prev-slot weighted-avg MCS;

$slots_since_under$: underutilization counter; $slots_since_update$: slots since last SM change

Ensure: PUSCH grants within SM compute budget; SM envelope updated for next slot

```
1: // Entry-point initialisation
2:  $tgt\_sm \leftarrow sm\_prev$ ;  $mcs\_est \leftarrow mcs\_prev$ ;  $iter \leftarrow 0$ ;  $high\_demand\_seen \leftarrow false$ 
3: // Iterative scheduling pipeline (Fig. 8, steps a-c)
4: PIPELINEENTRY:  $iter \leftarrow iter + 1$ 
5: // Step (a): PRB Budgeting
6:  $prbs\_avail \leftarrow LUT\_PRB(mcs\_est, tgt\_sm)$  ▷ max PRBs decodable within slot deadline
7: // Step (a'): Standard PUSCH Scheduling within  $prbs\_avail$ 
8: Run standard PF scheduling capped by  $prbs\_avail$ 
9: Record  $prbs\_alloc \leftarrow \sum_u rbSize_u$ ,  $sum\_backlog \leftarrow \sum_u B_u$ 
10:  $mcs\_curr \leftarrow \sum_u mcs_u \cdot rbSize_u / prbs\_alloc$ 
11: // Step (b): Adaptive SM Count Adjustment
12: // Ramp-Up: immediate increase to drain backlog
13: if  $\exists u$  with  $B_u > BACKLOG\_THR$  and the PRB budget is exhausted or residual backlog exceeds threshold then
14:    $est\_prbs \leftarrow \max(prbs\_alloc + min\_rb, est\_prbs\_from\_backlog(sum\_backlog))$ 
15:    $tgt\_sm \leftarrow LUT\_SM(mcs\_est, est\_prbs)$ ;  $high\_demand\_seen \leftarrow true$ 
16: end if
17: // Fast Exit: immediate down-sizing when slot is idle
18: if not  $high\_demand\_seen$  and  $prbs\_alloc = 0$  and  $tgt\_sm > HIGH\_TIER\_CAP$  then
19:    $exit\_sm \leftarrow LUT\_SM(mcs\_est, est\_prbs\_from\_backlog(sum\_backlog))$ 
20:   if  $exit\_sm \leq HIGH\_TIER\_CAP$  then
21:      $tgt\_sm \leftarrow \max(tgt\_sm - STEP, \max(exit\_sm, SM\_MIN))$  ▷ step down one tier
22:   end if
23: end if
24: // Hold or Ramp-Down: hysteresis-based conservative step-down
25: if  $tgt\_sm \neq sm\_prev$  then
26:    $slots\_since\_under \leftarrow 0$ ;  $slots\_since\_update \leftarrow 0$ 
27: else
28:    $slots\_since\_update += 1$ 
29:   if  $sm\_prev > SM\_MIN$  and  $LUT\_PRB(mcs\_curr, sm\_prev - STEP) \geq prbs\_alloc$  then
30:      $slots\_since\_under += 1$  ▷ one fewer tier would have sufficed
31:   else
32:      $slots\_since\_under \leftarrow 0$ 
33:   end if
34: end if
35: if  $slots\_since\_under \geq DECR\_THR$  then
36:    $desired\_sm \leftarrow \max(LUT\_SM(mcs\_est, est\_prbs\_from\_backlog(sum\_backlog)), SM\_MIN)$ 
37:    $tgt\_sm \leftarrow \max(sm\_prev - STEP, desired\_sm)$ ;  $slots\_since\_under \leftarrow 0$ 
38: end if
39: // Step (c): Iteration Control
40:  $P_{max} \leftarrow LUT\_PRB(mcs\_curr, tgt\_sm)$  ▷ recompute with true MCS and updated SM
41: if  $prbs\_alloc \leq P_{max}$  and  $P_{max} - prbs\_alloc \leq PRB\_TOL$  then
42:   SUCCESS: emit grants
43: else if  $iter < MAX\_ITER$  then
44:    $mcs\_est \leftarrow mcs\_curr$ ; goto PIPELINEENTRY
45: else if  $prbs\_alloc > P_{max}$  then
46:   SAFEGUARD: force  $tgt\_sm \leftarrow LUT\_SM(mcs\_curr, prbs\_alloc)$  ▷ ramp-up to guarantee forward progress
47: else
48:   CONSERVATIVE EXIT: emit grants ▷ safe but under-allocated after hitting the iteration bound
49: end if
50: // Carry state to next slot
51:  $sm\_prev \leftarrow tgt\_sm$  ▷ commit SM envelope for co-located workloads
52: if  $mcs\_curr > 0$  then
53:    $mcs\_prev \leftarrow mcs\_curr$ 
54: end if
55: Output: final UL PRB grants and  $tgt\_sm$  for this slot
```

response to backlog, how cautiously it ramps down after demand subsides, and how tightly iteration control tracks the recomputed PRB allowance. We choose their values through

end-to-end profiling on the DGX Spark testbed across different traffic scenarios. The tuning process sweeps each parameter while measuring SM-envelope stability (transition

Table 8: Compute-aware scheduler parameters.

Symbol	Description	Value
<i>SM_MIN</i>	Minimum SM count (floor)	8 SMs
<i>SM_MAX</i>	Maximum SM count (all SMs)	48 SMs
<i>STEP</i>	SM increment/decrement step	8 SMs
<i>BACKLOG_THR</i>	Per-UE backlog ramp-up threshold	64 KB
<i>DECR_THR</i>	Ramp-down patience (slots)	30 slots
<i>PRB_TOL</i>	PRB slack tolerance in step (c)	10 PRBs
<i>MAX_ITER</i>	Pipeline iteration bound	5

count), HARQ deadline-met rate, and UE-level KPIs (throughput, delay, and loss), then selects values that improve the envelope stability without degrading RAN performance.

SM_MIN and STEP. define the scheduler’s control granularity. The SM count is quantized in steps of 8, matching the CUDA Green Context co-scheduling granularity observed on the DGX Spark. Setting the minimum to 8 SMs ensures that even the lightest RAN workload can still be decoded within the slot deadline.

BACKLOG_THR. governs when the scheduler switches from smoothing to draining. We set it to 64 KB, approximately half of a full-bandwidth transport block at moderate MCS. A smaller value triggers ramp-up more eagerly, improving UE latency but causing more SM transitions; a larger value absorbs more traffic bursts at a constant SM tier, reducing transitions at the cost of slightly higher buffer occupancy.

DECR_THR. governs how much evidence of over-provisioning is required before releasing SMs. We set the ramp-down patience to 30 consecutive slots (~ 15 ms at 30 kHz SCS). The scheduler therefore steps down only after sustained over-provisioning, preventing premature SM release from fragmenting the envelope. This value is chosen to exceed the typical burst duration observed in the RAN.

PRB_TOL. bounds how much PRB under-allocation is tolerated in iteration control. We set it to 10 PRBs, so the scheduler accepts small slack gaps without another pass.

MAX_ITER. bounds the iteration loop. We cap the pipeline at 5 iterations per slot. In practice, it converges in 1–3 iterations in $>99\%$ of slots; this bound is only a safeguard against non-convergence under extreme channel conditions or workloads.

C WEAVER Training Strategy

C.1 Scheduling Mechanism Details

This subsection provides additional scheduling-mechanism details that complement the main training strategy in §7.

Coordinator metadata. The coordinator maintains three types of runtime state: (i) *executor performance state*, recent throughput and latency observations used to predict micro-batch completion times and decide when to trigger load

rebalancing; (ii) *task ownership state*, per-sample identifiers that ensure each training sample contributes exactly once to gradient aggregation even when work is rerouted or recomputed; and (iii) *deferral statistics*, per-step deferral rates that distinguish transient overload from persistent imbalance and drive the MLR trigger.

Rebalancing protocol. When MLR fires, the slow path shifts the pipeline-parallel partition boundary so that persistently overloaded stages shed layers to less-loaded sites, rather than migrating the full runtime data path. Only the persistent state for layers that cross the boundary (parameters and optimizer state) is transferred; transient activations are regenerated under the new layout. Resharding pauses training at a barrier boundary for the duration of state transfer, typically seconds, which is amortized over the hours-long intervals between layout changes. The underlying PREPARE/-COPY/COMMIT protocol ensures that layout transitions are consistent and do not corrupt gradients.

Eager preparation implementation. At worker initialization, WEAVER prepares a set of SM-partitioned execution streams that can be activated according to the current RAN SM envelope. WEAVER pre-creates CUDA Green Context streams for valid SM partitions together with associated memory bookkeeping, so runtime switching becomes a lightweight context rebinding operation rather than on-demand resource allocation. Each stream occupies 20 MB of GPU memory, and in total up to 16 streams are needed, with a swap overhead below $10\ \mu\text{s}$ compared to over 100 ms for on-demand Green Context creation. This eager preparation shifts almost all resource management out of the slot-critical path, so the only runtime operation is choosing which Green Context to bind a tile to it. These behaviors allow a worker to track and react to SM envelope changes on the order of microseconds while still presenting millisecond-scale execution units to higher layers.

Tile admission and profiling. To avoid runtime jitter, WEAVER pre-allocates GPU task buffers and stream state during initialization, eliminating online allocator overhead (100–300 μs under load) from the tile-admission path. Before deployment, WEAVER profiles GEMM performance across tile sizes and SM envelopes to build a lookup table that maps tile size $\times$ SM envelope to predicted completion time, so runtime tile admission can enforce the deadline and bandwidth constraints without per-decision profiling.

Fault tolerance. WEAVER handles crash-stop site failures through the same mechanisms used for capacity fluctuations. When a site becomes unreachable, the coordinator detects the failure via lease timeout, revokes the failed site’s ownership epoch, and reassigns its pending micro-batches to surviving replicas through AIR’s min-cost assignment (??).

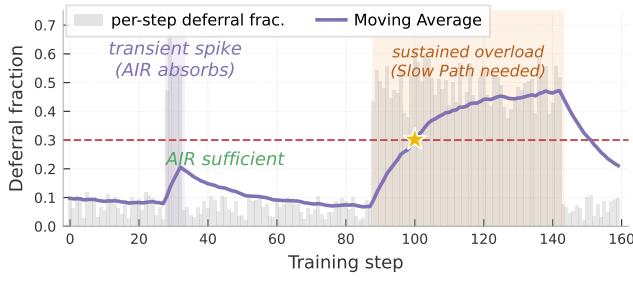


Figure 14: MLR trigger based on AIR deferrals. ★ refers to rebalancing trigger event.

Fenced ownership guarantees exact-once semantics: any delayed output from the failed site carries a stale epoch and is rejected at commit (§C.4, Claim 2). For persistent site loss, MLR rebalances the model layout to exclude the unavailable site, transferring only the affected partition’s optimizer state. We do not address Byzantine faults, correlated failures across a majority of sites, or coordinator crash recovery; the coordinator is assumed to be a reliable service, deployable with standard replication techniques if higher availability is required.

MLR example. Fig. 14 illustrates how MLR distinguishes transient overload from persistent imbalance. Initially, deferrals remain low and the moving average stays well below the threshold, so AIR alone is sufficient. Later, deferrals remain high over many consecutive steps; the moving average climbs above the threshold and stays there, at which point the system invokes the slow path to adjust the model parallelism layout. MLR only reacts when overload is sustained, avoiding expensive rebalancing (needs state migration) for short-lived spikes that AIR can handle on its own.

C.2 Correctness Guarantees

Correctness for WEAVER is defined relative to a reference run that trains the same model with standard synchronous data-parallel SGD on a fixed cluster. Dynamic rerouting, deferral, and pipeline resharding should not change which samples are counted or how their gradients are combined, they should only change where and when the work runs.

No duplicate or stale execution. Each micro-batch carries a metadata tuple that includes its step t , sample id u , stage s , and a fenced ownership token. The coordinator maintains a ledger keyed by (t, u, s) and only accepts commits from the current owner. AIR rerouting and Level 2 resharding both use the same transfer handshake, so late outputs from old owners or previous layout epochs are rejected rather than applied twice. As a result, each sample contributes at most once per step and no gradient is applied from a stale model version.

Per-sample gradient correctness with variable participation. Because Level 1 can defer work and replicas may complete different counts $n_{g,t}$ in a step, the effective batch size B_t varies over time and participation across replicas is uneven. WEAVER aggregates gradients using the weighted rule in Eq. (2), which uses the actual counts $n_{g,t}$ from each replica instead of assuming fixed quotas. This produces the exact per-sample average over the set of samples that complete in step t , matching what a reference synchronous run would compute on that same sample set. The debt update in Eq. (4) then adjusts future quotas when a replica under-contributes for many steps, which keeps long-run participation balanced without changing the per-step estimator. §C.4 gives the convergence argument under this variable effective batch size.

Bounded latency and eventual progress from two-level adaptation. AIR enforces a step-closing horizon around the barrier by admitting only micro-batches whose predicted finish time fits within the elastic window exported by RAN Control. Micro-batches that would overrun this horizon are deferred before they acquire ownership, which preserves exact-once semantics while preventing unbounded queuing. When capacity shrinks enough that Level 1 would keep deferring a large fraction of work, the Level 2 trigger adjusts pipeline boundaries and quotas until the new layout again fits within the same horizon. Together, these two levels ensure that training either makes bounded-latency progress at the current layout or quickly transitions to a new layout that does, so the system avoids both deadline blow-ups and starvation.

C.3 Scheduling Formalism

This subsection collects the formal latency, aggregation, debt, and control-loop definitions used by the two-level scheduler in §7. We keep the same notation as the main text and use these equations as the canonical reference for proofs and implementation details.

AIR enforces a step-closing horizon around barrier deadline T_{bar} with bounded elastic slack $\Delta_t \leq \Delta_{\text{max}}$. Admitting only micro-batches that finish within this horizon yields the step-latency bound

$$L_t \leq T_{\text{bar}} + \Delta_{\text{max}} + \Delta_c, \quad (1)$$

where L_t is observed step latency, Δ_{max} is the configured maximum elastic slack, and Δ_c is the online control period exported by RAN Control.

After rerouting/deferral, replica g may complete $n_{g,t}$ samples at step t , with summed local gradient contribution $G_{g,t}$. The tree-reduce through the hub computes

$$\bar{g}_t = \frac{\sum_{g=1}^G G_{g,t}}{\sum_{g=1}^G n_{g,t}}. \quad (2)$$

Here $\bar{g}_t$ is the global per-sample average over all completed exact-once samples in step t , and G is the number of participating replicas in that step.

To correct persistent contribution skew, WEAVER maintains per-replica debt state d_g and updates next-step quota q_g using

$$d_g(t+1) = d_g(t) + (n^{\text{target}} - n_{g,t}), \quad (3)$$

$$q_g(t+1) = \text{clip}\left(q_g^{\text{base}} + \kappa d_g(t+1)\right). \quad (4)$$

where n^{target} is the target per-replica completion count, q_g^{base} is the baseline quota from placement, and κ is the debt-correction gain.

The online scheduler runs a 10–50 ms control loop that combines latency prediction, AIR routing, exact-once accounting, weighted aggregation, debt update, and optional Level-2 resharding at barrier boundaries:

C.4 Proof Sketches for AIR Correctness

This section provides proof sketches for the four claims in §7.

Claim 1 (weighted aggregation is unbiased). For step t , let S_t be the set of completed (accepted) samples after rerouting and deduplication. For each sample $i \in S_t$, let $g_i(w_t)$ denote its per-sample gradient at synchronized weight w_t . Replica g contributes

$$G_{g,t} = \sum_{i \in S_{g,t}} g_i(w_t), \quad n_{g,t} = |S_{g,t}|, \quad (5)$$

with disjoint partition $S_t = \bigsqcup_g S_{g,t}$ due to exact-once acceptance. The estimator in Eq. (2) can be rewritten as

$$\bar{g}_t = \frac{\sum_g G_{g,t}}{\sum_g n_{g,t}} = \frac{1}{|S_t|} \sum_{i \in S_t} g_i(w_t). \quad (6)$$

Hence it is exactly the sample average over all completed samples. Under standard random mini-batch sampling and the assumption that admission/defer/reroute decisions are independent of sample content conditioned on the system state, $\mathbb{E}[\bar{g}_t \mid w_t]$ equals the unbiased gradient of the completed-sample objective for that step.

Claim 2 (exact-once under fenced ownership). Each stage key is $\chi = (t, u, s)$ with ledger state (o, e, τ) : owner o , epoch e , token τ . Ownership transfer is a CAS update on (o, e) :

$$(o, e) \xrightarrow{\text{CAS}} (o', e+1), \quad (7)$$

followed by token fencing (old token revoked, new token activated).

Safety (no duplicates): Suppose two commits for same key χ are accepted. Then either (i) same epoch e from two owners, impossible because CAS defines a unique owner for each epoch, or (ii) different epochs $e < e'$, but commit validation

accepts only current epoch/token and rejects stale epochs. Contradiction.

Liveness (progress with failures): If owner fails, lease timeout triggers reassignment with higher epoch. Any delayed old-owner output is stale and rejected. Therefore, if some healthy executor eventually acquires ownership and completes, at most a single accepted commit is recorded.

Claim 3 (debt rebalance bound). Define debt dynamics (Eq. (4)) and let realized count be

$$n_{g,t} = n^{\text{target}} + \kappa d_g(t) + \xi_{g,t}, \quad (8)$$

where $\xi_{g,t}$ captures bounded actuation/scheduling noise, $|\xi_{g,t}| \leq \Xi$. Then

$$d_g(t+1) = (1 - \kappa)d_g(t) - \xi_{g,t}. \quad (9)$$

For $0 < \kappa < 1$, this is a stable linear system with bounded disturbance. Recursing gives

$$|d_g(t)| \leq (1 - \kappa)^t |d_g(0)| + \frac{\Xi}{\kappa}. \quad (10)$$

Thus long-run skew is bounded by $O(\Xi/\kappa)$ and decays geometrically when noise is small.

Claim 4 (reshard consistency). Reshard events occur only at a global barrier boundary t^* where the pipeline is quiescent at the changing boundary. In the *PREPARE* phase, WEAVER freezes AIR assignments whose path would cross the moved layers, drains all in-flight work under the old layout, and stops admitting new work at the affected stages while the rest of the pipeline continues. In the *COPY* phase, it transfers state S_t for the moved layers and verifies checksums without changing ownership metadata. In the *COMMIT* phase, it atomically publishes the new boundary vector, rebuilds equivalent-stage pools $\{\mathcal{E}_s\}$, and increments the monotonic layout epoch r in the coordinator ledger. After COMMIT, any new micro-batch that enters the affected stages carries the updated epoch in its metadata $\langle t, u, s, o, e, v, \tau, r \rangle$, and the coordinator accepts a completion for key $\chi = (t, u, s)$ only if its (e, τ, r) match the current ledger entry. Hence any accepted completion for χ is executed entirely under a single pipeline layout identified by its layout epoch.

Extending the fenced-ownership check from Claim 2 to include r preserves exact-once accounting across boundary moves. Suppose two distinct commits for the same key χ were both accepted around a reshard event. Either they carry the same (e, τ, r) , which contradicts fenced ownership and CAS uniqueness in Claim 2, or they carry different layout epochs $r < r'$. In the latter case, validation against the ledger accepts only the current layout epoch and lease, so any output with stale (e, τ) or stale r is rejected. Thus at most one commit for each χ is ever accepted, and no commit is recorded with mixed or stale layout information.

Protocol invariant summary. Taken together, Claims 1–4 establish one end-to-end acceptance invariant per stage key

$\chi = (t, u, s)$: (i) metadata-gated validation on $\langle t, u, s, o, e, v, \tau, r \rangle$ enforces version and layout compatibility before admission; (ii) fenced ownership with CAS on (o, e) allows at most one accepted commit for each key; (iii) PREPARE/COPY/COMMIT performs an atomic cutover at step boundary t^* , so each accepted completion is bound to exactly one layout epoch r ; and (iv) outputs with stale (e, τ) or stale r are rejected, while reclamation is deferred until drain completion, preserving safe cleanup without violating exact-once accounting.

C.5 Performance Justifications

Control-plane overhead. The coordinator’s scheduling decisions execute on CPU at 10–50 ms granularity and do not contend with GPU-resident training or RAN kernels. Per-step coordinator overhead is 120 μ s, accounting for <0.3% of step time, confirming that the distributed scheduling control plane is lightweight.

Correctness and convergence. Four properties hold despite dynamic capacity. *First*, exact-once sample accounting: every in-flight micro-batch carries an owner record, and ownership transfer is fenced whenever the work moves between executors. The coordinator allows at most one owner to commit and rejects stale or duplicate commits. *Second*, strong synchronization: resharding runs only at barrier boundaries via PREPARE/COPY/COMMIT, so step-level synchronization matches fixed-layout synchronous SGD. *Third*, bounded gradient debt: the debt-rebalance dynamics remain stable for $0 < \kappa < 1$, giving a bounded long-run skew and geometric decay under bounded scheduling noise (Claim 3). *Fourth*, unbiased gradient aggregation: the per-step gradient average is weighted by actual per-replica sample counts, so rerouting changes *when* a sample contributes but not *how* its gradient is weighted once committed.

D Evaluation Details

D.1 RAN Centric Spare Compute Controller Evaluation

D.1.1 RAN configuration. The gNB runs 5G NR standalone (SA) on band n78 (3.5 GHz TDD) with 106 PRBs at 30 kHz sub-carrier spacing (numerology 1), corresponding to a 40 MHz carrier. The TDD pattern uses a 5 ms periodicity with 7 DL slots, 1 flexible slot, and 2 UL slots (DDDDDDDSUU), yielding a $\sim 20\%$ UL duty cycle. All experiments run on an NVIDIA DGX Spark with 48 SMs.

D.1.2 Channel condition replay. Static AWGN channels represent the primary fidelity gap between RF-simulated and OTA experiments. We close this gap by replaying per-UE uplink channel measurements from the TRACTOR dataset [20] into OAI RFSIM’s channel model. At each trace timestamp,

the recorded UL RSSI and SINR are translated into the simulator’s path-loss and noise-power controls:

$$\text{ploss_dB}_{u,k} = P_{\text{ref}} - \text{RSSI}_{u,k}, \quad (11)$$

$$\text{noise_power_dB}_{u,k} = N_{\text{off}} - \text{SINR}_{u,k}, \quad (12)$$

where P_{ref} and N_{off} are fixed calibration offsets. Updates are sent via OAI’s telnet interface and synchronized with traffic generators using epoch timestamps, so each UE observes the same time-varying channel trajectory as the original trace.

D.1.3 Application traffic replay.

TRACTOR application trace replay. We use production O-RAN traffic from the TRACTOR 5G dataset [20], selecting two four-UE scenarios: Scenario 1 (referred to as *Multi-UE/Trial2/multi4* in the dataset) and Scenario 2 (referred to as *Multi-UE/Trial3/multi4* in the dataset). Per-UE traffic profiles and channel conditions are listed in Table 9. Traces are replayed with the original packet timing over a 600 s window; shorter UE traces are looped to ensure all four UEs remain active.

D-ITG stress traffic. For UE-level KPI evaluation, we use D-ITG [8] to generate synthetic bursty UDP traffic that exercises more extreme conditions than production traces. Each UE runs a steady 800 kbps flow plus a 20 s burst at 3.6 Mbps, so the aggregate peak load (~ 13.2 Mbps across three UEs) saturates the uplink.

D.1.4 HARQ deadline-met rate. After the gNB decodes an UL transport block, it must send HARQ ACK/NACK feedback to the UE within the K_1 -slot window specified by the PUCCH configuration [1]. GPU-accelerated LDPC decoding is the critical path in this window. We instrument the OAI MAC scheduler to track UL HARQ feedback timing: when a HARQ process is scheduled, the expected feedback frame and slot are recorded; at each subsequent scheduling round, we check whether the feedback has arrived within K_1 slots past the expected time. If it has not, the process is marked as a deadline miss. The HARQ deadline-met rate is then $1 - (\text{missed}/\text{total UL TBs})$ over the trace.

Table 9: TRACTOR dataset configurations used in RAN control evaluation.

Datasets	UE	Traffic profile	Type	Avg, UL SINR
Scenario 1	UE 1	Background traffic	mMTC	13.1 dB
	UE 2	YouTube streaming (stationary)	eMBB	2.3 dB
	UE 3	Background traffic	mMTC	3.3 dB
	UE 4	YouTube streaming (stationary)	eMBB	6.0 dB
Scenario 2	UE 1	Netflix streaming (stationary)	eMBB	20.7 dB
	UE 2	YouTube streaming (campus walk)	eMBB	5.5 dB
	UE 3	Background traffic	mMTC	4.2 dB
	UE 4	Internet browsing (stationary)	eMBB	12.2 dB

Table 13: Model-scaling throughput on 512 devices. Rescaled/Typical: 48 GB GPUs; Ideal: 144 GB GPUs. OOM = out-of-memory.

Model	System	Peak Mem (GB)	Throughput (tok/s)		
			Rescaled	Typical	Ideal
OPT-13B	WEAVER	13.8	92.2K	350.6K	484.8K
	Confidant*	13.9	47.6K	87.3K	182.8K
	Confidant	27.9	46.6K	49.7K	101.3K
OPT-30B	WEAVER	30.2	38.4K	146.1K	202.0K
	Confidant*	57.6	OOM	OOM	65.3K
	Confidant	70.0	OOM	OOM	4.8K
OPT-70B	WEAVER	40.2	15.4K	58.4K	80.8K
	Confidant*	75.2	OOM	OOM	22.9K
	Confidant	149.8	OOM	OOM	OOM

OOM indicates the model exceeds the available GPU memory budget.

Table 14: Impact of RAN control method on training. ↑/↓ means the higher/lower the better.

Method	Violations ↓	Efficiency ↑	Token/s ↑	TFLOPs ↑
YINYANGRAN	N/A	32%	1447	54
NoCTRL	2.7%	67%	3827	118
WEAVER	0.6%	83%	4088	142

Table 10: Inter-site bandwidth matrix (src_rank → dst_rank), in Gbps. Diagonal entries are not applicable (—).

src/dst	rank0	rank1	rank2	rank3	rank4	rank5	rank6	rank7
rank0	—	11.0	0.3	1.0	0.9	0.9	1.0	1.0
rank1	20.5	—	0.3	1.0	1.0	0.9	1.0	1.0
rank2	1.1	1.0	—	0.9	0.9	10.6	1.0	1.0
rank3	1.1	1.0	0.3	—	14.6	0.9	1.0	1.0
rank4	1.1	1.0	0.3	11.9	—	1.0	1.0	1.0
rank5	1.1	1.0	10.2	0.9	0.9	—	1.0	1.0
rank6	1.0	0.8	0.3	0.8	0.8	0.8	—	14.5
rank7	1.0	0.9	0.3	0.8	0.9	0.8	17.9	—

Table 11: Inter-node latency matrix (src_rank → dst_rank), in ms. Diagonal entries are not applicable (—).

src/dst	rank0	rank1	rank2	rank3	rank4	rank5	rank6	rank7
rank0	—	0.8	25.9	26.7	26.7	25.7	23.7	23.5
rank1	0.8	—	25.7	26.5	26.7	26.5	23.6	23.7
rank2	25.7	25.8	—	28.8	28.8	0.8	25.7	25.6
rank3	26.4	26.5	28.6	—	30.5	28.7	26.4	26.6
rank4	26.6	26.6	28.8	0.9	—	28.6	26.6	26.5
rank5	25.8	25.9	0.9	28.8	28.8	—	25.6	25.7
rank6	23.7	23.6	25.7	26.5	26.6	25.8	—	0.2
rank7	23.5	23.5	25.6	26.6	26.7	25.7	0.2	—

D.2 Training Evaluation Settings

Table 10 provides the inter-site bandwidth and Table 11 provides the inter-site latency.

D.3 Large Scale Evaluation of Training

Table 12 and Table 13 show the full model and device scaling result.

Table 12: Device-scaling throughput and achieved cluster FLOPS across three environments for OPT-13B.

Devices	System	Rescaled		Typical		Ideal	
		tok/s	FLOPS%	tok/s	FLOPS%	tok/s	FLOPS%
128	WEAVER	76.8K	12.6%	106.7K	17.7%	121.0K	56.1%
	Confidant*	47.6K	7.8%	58.5K	8.6%	107.7K	30.2%
	Confidant	46.6K	7.8%	46.5K	7.8%	101.3K	30.5%
256	WEAVER	87.8K	11.1%	198.8K	15.7%	242.1K	52.4%
	Confidant*	47.6K	4.6%	77.6K	22.4%	153K	30.5%
	Confidant	46.6K	4.6%	48.6K	4.8%	101.3K	18.7%
512	WEAVER	92.2K	10.8%	350.6K	12.7%	484.8K	51.8%
	Confidant*	47.6K	2.3%	87.3K	3.8%	182.8K	12.3%
	Confidant	46.6K	2.3%	49.7K	1.7%	101.3K	9.8%

D.4 Intra-Site Scheduling

We evaluate the effectiveness of RAN control in improving training performance at a single site. We compare three schemes: (i) YINYANGRAN for GPU MPS-based resource control; since their prediction and feedback loop is not available, we reconfigure MPS once per second to an *oracle* SM share as if future traffic were known; (ii) NoCTRL, which leaves MAC scheduling and SM allocation unchanged, with no SM envelope shaping and only the minimal SMs required per slot reserved for RAN; and (iii) the full RAN control component in WEAVER. Due to the GPU reconfiguration nature of MPS, we checkpoint training to host memory. All methods are evaluated using the SM envelope trace generated in §8.1.

All schemes observe identical traffic and share the same model and optimizer state; any performance differences therefore arise solely from the RAN control policy. We use OPT-13B (12 layers) with batch size 4 and sequence length 512, representing a single NVIDIA L40S training capacity, microbatch-based training configuration. We run the experiment for 2 min for statistical significance.

WEAVER guarantees low risk of isolation violation. We first look at the microscopic-level of the adaptation. We define an isolation *violation* as any slot where, at the start of LDPC decoding, training kernels have not fully complied with the SMs required by the RAN-configured allocation, thereby putting the decoding deadline at risk. This can occur when the RAN raises its SM cap for the current slot, but training kernels launched have not yet completed.

Under NoCTRL we observe 118 such violations with a total accumulated violation time of 41.5 ms across the trace. With WEAVER enabled, hysteresis and post-round correction reduce the count to 26 violations, achieving a 4.8× reduction in the number of violations and a 2.7× reduction in total duration. In other words, WEAVER maintains RAN isolation with only rare and short-lived excursions despite co-located training. Note that MPS does not comply with this criterion as GPU processing needs to be fully paused for RAN and training upon SM count reconfiguration.

Table 15: Level-2 reshard overhead decomposition per event, averaged over 11 triggers in the 10-minute trace. Values are preliminary and will be updated once the full trace audit is complete.

Phase	Total (ms)	Stall (ms)
PREPARE (drain)	528.0	528.0
COPY (transfer)	4350.6	0.0
COMMIT (activate)	550.0	0.0
Total	5428.6	528.0

WEAVER guarantees highly efficient compute utilization. We next examine how effectively each method harvests spare compute. We define *efficiency* as the fraction of ground-truth spare SM capacity that is actually used over time. As shown in Table 14, WEAVER achieves an efficiency of 83%, compared to 67% for NoCTRL and only 32% for MPS, indicating that active RAN control converts substantially more of the exposed slack into useful training work.

This higher efficiency translates directly into end-to-end training performance. Relative to NoCTRL, WEAVER increases token throughput and achieved TFLOPs by 1.3 $\times$, with the model, optimizer, and batch configuration held fixed. Aggregated over the trace, the total SM reconfiguration overhead accounts for only 0.0009% of training time, which is negligible compared to the gains in efficiency, throughput, and TFLOPs.

D.5 Inter-Site Scheduling

We now evaluate how the two-level inter-site scheduler in §7.1 behaves under replayed RAN traces. We feed a representative 10-minute production traffic trace through our training simulator on OPT-13B across 8 edge sites with NVIDIA L40S GPUs, so any difference comes only from the policies described in §7.1.

For the training side, we compare WEAVER against the same DTFM, Asteroid, and Confidant baselines described in §8.2, which all react at slow timescales while sharing the common oracle-MPS substrate.

Fast-path AIR absorbs most capacity fluctuations. Fig. 15 summarizes how Asynchronous Inter-site Rerouting behaves when driven by real RAN traces. Over the 10-minute window, spare compute fluctuates 80 times at the per-step timescale (2–8 s). Of these events, 86.2% are handled entirely by Level-1 AIR rerouting with a median exposed stall of 66 ms (P95: 68 ms), and only 11 events escalate to the slow path. This matches the intent of the AIR design in §7.1: the coordinator moves work, not state, so most transient drops in spare compute are hidden behind normal training progress.

The runtime view in Fig. 15 also shows that AIR decisions occur within the current training step rather than waiting for a global barrier. The coordinator reassigns pending micro-batches to equivalent-stage executors in other data-parallel replicas, with no GPU reconfiguration, no MPS reset, and no checkpoint replay. As a result, the RAN subframe deadline is not endangered even when site capacity swings quickly, which is the safety property targeted by the inter-site scheduler in §7.1.

Slow-path resharding is rare and low-overhead. When spare compute drifts in a persistent way, Level-1 AIR starts to defer a growing fraction of micro-batches. Each deferral is still within the bound enforced by the inter-site policy, but the accumulated signal in the EWMA trigger state z_t eventually crosses the reshard threshold and activates Level-2 resharding.

In Fig. 15, the middle panel tracks z_t and the reshard threshold over time. Across the 10-minute trace, the Level-2 trigger fires 11 times, which confirms that the deferral-based trigger in §7.1 reacts only when imbalance is sustained rather than to short spikes.

To make the cost of each reshard event concrete, Table 15 decomposes the Level-2 protocol into its phases. These numbers come from the same trace replay used above and are reported here to guide the choice of reshard thresholds; they remain subject to further validation.

The PREPARE phase is the only component on the critical path, and its 528 ms stall cost is amortized over many training steps between reshard events. COPY and COMMIT mostly overlap with useful work on neighboring stages. Taken together with the 11-event count above, this supports the design choice in §7.1 to keep layout changes rare and hiding most of their cost behind ongoing execution.

Inter-site scheduling parameter sensitivity. Finally, we study how the thresholds that drive AIR and MLR affect throughput and stability under the same replayed envelopes used above. We sweep the Level-1 slack threshold θ_{AIR} and the Level-2 trigger threshold θ_{MLR} while keeping all other settings fixed.

Across these settings, throughput varies only slightly while the CoV remains well below 1%, which indicates stable step latency. More aggressive MLR triggering increases the number of reshard events but does not improve throughput in a meaningful way. This backs the design in §7.1: a conservative trigger that lets AIR handle most transients and invokes MLR rarely is enough to keep the system both stable and efficient.

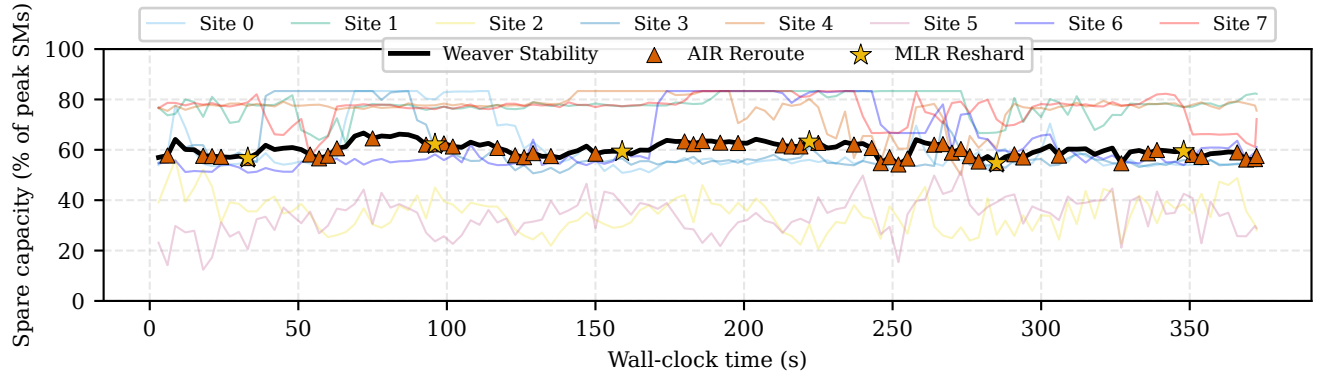


Figure 15: Timeline of inter-site scheduling over a 10-minute replay window. The top panel will show spare compute and step latency per site, the middle panel the EWMA trigger state z_t and reshard threshold, and the bottom panel the Level-2 reshard protocol windows.

